



DEMOCRITUS
UNIVERSITY
OF THRACE

DEPARTMENT OF
ELECTRICAL & COMPUTER
ENGINEERING

MSc in QUANTUM
COMPUTING AND
QUANTUM
TECHNOLOGIES



NATIONAL CENTRE FOR
SCIENTIFIC RESEARCH "DEMOKRITOS"

M.Sc. in Quantum Computing and Quantum Technologies

Democritus University of Thrace, Department of Electrical and Computer
Engineering

Quantum Machine Learning and Applications

Postgraduate Diploma (Msc) Dissertation

Asterios D. Pantousas 60637

Supervisor:

Pavlos S. Efraimidis

October 6, 2025

Contents

1	Introduction	8
2	Background and Literature Review	10
2.1	Introduction	10
2.2	Machine Learning	10
2.2.1	Key Concepts	10
2.3	Quantum Computing	11
2.3.1	Key Concepts	11
2.4	Quantum Machine Learning(QML)	13
2.4.1	How Quantum Machine Learning works	13
2.4.2	Key Algorithms in QML	15
2.4.3	Quantum Machine Learning Libraries	16
2.4.4	Advantages and Challenges of QML	17
2.4.5	Recent Developments and Applications	18
2.5	Theoretical Comparison Between Classical and Quantum Machine Learning	18
2.5.1	Model-Level Analogies Between Classical and Quantum ML	19
3	Methodology - My Quantum (and Classical) Quest	21
3.1	Datasets Description: The Training Grounds (A Deeper Dive)	21
3.1.1	Heart Failure Prediction Dataset: The Heart of the Matter	21
3.1.2	Stroke Prediction Dataset: Brain Games	22
3.1.3	The Big Picture	23
3.2	Classical Machine Learning Models: Our Baseline Arsenal	23
3.2.1	Logistic Regression: The Linear Linebacker	24
3.2.2	Naive Bayes: The Probabilistic Powerhouse (with a Big Assumption)	25
3.2.3	Support Vector Machines (SVM) - Linear Kernel: The Margin Maximizer	26
3.2.4	Support Vector Machines (SVM) - RBF Kernel: The Non-Linear Navigator	28
3.2.5	K-Nearest Neighbors (KNN): The Neighborhood Knower	29
3.3	Quantum Machine Learning Models: Entering the Quantum Realm	30
3.3.1	Variational Quantum Classifier (VQC): The Quantum Gymnast	31
3.3.2	QSVM Pegasos: The Quantum Kernel Crusader	32
3.3.3	Sampler/Estimator QNN: The Hybrid Harmony	33
3.3.4	Evaluation Metrics: Judging the Quantum Jockeys	34
4	Code Implementation- Setting Up The Arena	36
4.1	Import Python Libraries	36
4.2	Setting Up the Environment	37
4.3	Configuring the Script	38
4.4	Setting Up IBM Quantum Service	39
4.5	Data Preprocessing	39

4.5.1	Loading the Dataset	40
4.5.2	Handling Missing Values	40
4.5.3	Encoding Categorical Features	40
4.5.4	Detecting Class Imbalance	41
4.5.5	Scaling Numerical Features	41
4.6	Evaluating Classical Machine Learning Models	41
4.6.1	Workflow of Model Evaluation	41
4.6.2	Python Code for Model Evaluation	42
4.7	Quantum Machine Learning Implementation Analysis	43
4.7.1	Setting the Quantum Stage	43
4.7.2	Choosing the Quantum Backend	43
4.7.3	Data Splitting and Model Evaluation	44
4.7.4	QML models analysis	45
4.7.5	Quantum Neural Networks (QNNs)	53
4.7.6	Technical Challenges and Considerations	56
4.7.7	Code and Reproducibility	58
4.7.8	Summary	58
5	Results and Discussion	59
5.1	Introduction	59
5.2	Experimental Setup	59
5.2.1	Models	59
5.2.2	Evaluation Metrics	60
5.2.3	Implementation Details	60
5.3	Results	60
5.3.1	Heart Failure Prediction	60
5.3.2	Stroke Prediction	69
5.4	Comparative Analysis	76
5.5	Technical Challenges	77
5.6	Summary	77
6	Conclusion and Future Work	78
	Bibliography	80

List of Figures

2.1	Quantum circuit implementing a simple entanglement operation with Hadamard and CNOT gates followed by measurement.	13
2.2	Overview of quantum machine learning approaches. Image by Schuld et al., licensed under CC BY-SA 4.0, via Wikimedia Commons (https://commons.wikimedia.org/wiki/File:Qml_approaches.tif).	13
2.3	Hybrid Quantum Machine Learning circuit for Variational quantum classifiers (VQCs)	14
3.1	Simple Logistic Regression implementation	25
3.2	Simple Naive Bayes implementation	26
3.3	Simple Support Vector Machine (Linear) implementation	28
3.4	Simple Support Vector Machine (RBF) implementation	29
3.5	Simple K Nearest Neighbors implementation	30
4.1	ZZFeatureMap for 3 features and single repetition	46
4.2	RealAmplitudes for 3 features and 3 repetitions	48
4.3	The paradigm of classical and quantum kernels [12]	51
4.4	Generic Quantum Neural Network (QNN) structure [34]	53
5.1	Accuracy comparison of classical and quantum models across different feature counts for the Heart Failure Prediction dataset.	66
5.2	F1-score trends for classical and quantum models as a function of feature count in the Heart Failure Prediction dataset.	66
5.3	Elapsed training/prediction time (log scale) comparison between classical and quantum models across different feature counts in the Heart Failure Prediction dataset.	67
5.4	Accuracy comparison of classical and quantum models across different feature counts for the Stroke Prediction dataset.	74
5.5	Elapsed time (log scale) for classical and quantum models as the number of features increases in the Stroke Prediction dataset.	75
5.6	F1-score trends for classical and quantum models as a function of feature count in the Stroke Prediction dataset.	75

List of Tables

2.1	Side-by-side analogy between classical and quantum ML models and mechanisms.	20
3.1	Snapshot of the Heart Failure Prediction Dataset	22
3.2	Snapshot of the Stroke Prediction Dataset	23
4.1	Comparison of Quantum Feature Maps	47
5.1	Dataset brief information	59
5.2	Comparison of Classical and Quantum Models (2 Features)	61
5.3	Comparison of Classical and Quantum Models (3 Features)	61
5.4	Comparison of Classical and Quantum Models (4 Features)	62
5.5	Comparison of Classical and Quantum Models (5 Features)	62
5.6	Comparison of Classical and Quantum Models (6 Features)	63
5.7	Comparison of Classical and Quantum Models (7 Features)	63
5.8	Comparison of Classical and Quantum Models (8 Features)	64
5.9	Comparison of Classical and Quantum Models (9 Features)	64
5.10	Comparison of Classical and Quantum Models (10 Features)	65
5.11	Comparison of Classical and Quantum Models (11 Features)	65
5.12	Comparison of Classical and Quantum Models for Stroke Prediction (2 Features)	69
5.13	Comparison of Classical and Quantum Models for Stroke Prediction (3 Features)	70
5.14	Comparison of Classical and Quantum Models for Stroke Prediction (4 Features)	70
5.15	Comparison of Classical and Quantum Models for Stroke Prediction (5 Features)	71
5.16	Comparison of Classical and Quantum Models for Stroke Prediction (6 Features)	71
5.17	Comparison of Classical and Quantum Models for Stroke Prediction (7 Features)	72
5.18	Comparison of Classical and Quantum Models for Stroke Prediction (8 Features)	72
5.19	Comparison of Classical and Quantum Models for Stroke Prediction (9 Features)	73
5.20	Comparison of Classical and Quantum Models for Stroke Prediction (10 Features)	73

Abstract

At a time when everyone is trying to ride the quantum wave, this thesis asks the critical question: Is Quantum Machine Learning actually useful, or just a fancy buzzword? Quantum Machine Learning (QML) emerges as a field where the seemingly bizarre laws of quantum mechanics meet the practical world of machine learning algorithms. We dive into the heart of QML, comparing it against classical Machine Learning (ML) algorithms to see if it can truly deliver what it promises. By implementing and testing both classical and quantum models using the Python language, Scikit Learn and Qiskit libraries, we aim to provide a more technical perspective of QML's potential, limitations, and whether it is worth the hype. We will be judging performance based on key metrics such as accuracy, precision, recall, and computational efficiency. While acknowledging current hardware limitations, we will explore QML's potential applications in healthcare sector. Spoiler alert: Reality may not be as entangled as we would like.

Keywords: Quantum Machine Learning, Classical Machine Learning, Qiskit, Superposition, Entanglement, Quantum Algorithms, Hybrid Models, Healthcare.

Acknowledgments

First and foremost, I owe a great debt of gratitude to my supervisor, Professor Pavlos Efraimidis. I am fairly certain that I tested the limits of his patience with my numerous delays. Thank you for your guidance, support and for not giving up on me, even when I probably deserved it. Your expertise and understanding were invaluable in bringing this thesis to completion.

Last but not least, I would like to thank my parents, who not only supported me financially through this Masters of Science degree but also demonstrated saint-like patience as I navigated the dark waters of thesis writing. Their unwavering belief in my abilities (even when I doubted them myself) kept me motivated. I am pretty sure they deserve an honorary degree in Quantum Computing at this point.

Chapter 1

Introduction

Quantum machine learning (QML) has emerged as a promising field that combines the principles of quantum mechanics and the power of machine learning algorithms. This innovative approach has the potential to unlock new frontiers in various domains, including data analysis, pattern recognition, and optimization problems. The integration of quantum computing and machine learning can lead to significant advancements in artificial intelligence, with the ability to tackle complex problems that are beyond the reach of classical computers [1].

The growing demand for faster computational power and the limitations that classical computing has, motivated the exploration of new paradigms such as quantum computing. Classical computers are constrained by Moore's Law which speculates that the transistors are reaching their physical limit because they are shrinking to atomic scales [2]. Quantum computing presents an opportunity to bypass these limitations by leveraging quantum principles like superposition, entanglement and interference. In addition to this, the current digital age is characterized by major advancements in the field of Artificial Intelligence due to the vast amount of data being collected and stored and the improvements in the computational power of classical computers [3]. The motivation behind this work is to explore whether QML can enhance existing machine learning models and provide solutions to computational intensive problems in the field of healthcare.

Significant contributions have already been made in both machine learning and quantum computing. Pioneers like Richard Feynman who proposed that quantum mechanics could be used in simulating physical systems more efficiently than classical computers [4] and David Deutsch who formalized the concept of a universal quantum computer that may solve problems exponentially faster by using superposition and entanglement [5]. While classical machine learning models like neural networks and decision trees had great success in tasks like image classification, natural language processing and robotics, recent research has shown the potential of quantum machine learning to outperform classical algorithms under certain conditions [6]. An interesting example is the collaboration between IonQ and Hyundai to advance self-driving car technology. They successfully trained a quantum machine learning image recognition algorithm using just 8 qubits, enabling the system to recognize common road signs with fewer input parameters compared to traditional classical approaches [7].

This thesis aims to investigate the potential of quantum algorithms in enhancing machine learning models. Even though the quantum machine learning field is in its early days, this research seeks to understand the potential advantages and limitations of quantum approaches by comparing classical machine learning models with quantum machine learning models using the same dataset. The purpose is not only to evaluate the theoretical benefits of quantum machine learning but also to test their practical viability with real-world data.

The methodology followed in this thesis consists of several stages. First, a thorough review of existing literature on classical machine learning and quantum computing will be conducted. This will be followed by the implementation of both classical and quantum machine learning models

using the Python programming language, alongside with libraries like Sklearn and Qiskit for quantum models [8]. Then, the quantum machine learning models will also be trained in a real-quantum computer in order to have a clear view of the current state of the field. Finally, the models will be tested on the same datasets and performance will be evaluated based on metrics like accuracy, precision, recall and computational efficiency. The comparison will provide insights into the potential benefits and limitations of QML.

Preliminary results suggest that quantum machine learning holds promise for specific types of tasks, especially those involving high-dimensional data and complex optimization problems. Specifically, the results from this research show that quantum machine learning models can achieve similar results with their classical alternatives using less data. Of course the current hardware limitations pose challenges to fully realizing the benefits of QML. The findings will be discussed in greater detail in later chapters, where the performance of classical and quantum machine learning models will be analyzed and compared.

The structure of this thesis is organized as follows. Chapter 2 will present a comprehensive literature review, covering the foundational concepts of classical machine learning, quantum computing, and quantum machine learning, as well as a theoretical comparison between classical and quantum machine learning approaches. Chapter 3 will detail the methodology used in this study, including a thorough description of the datasets, the architectures of both classical and quantum models, and the evaluation metrics selected for performance assessment. Chapter 4 will focus on the code implementation, presenting the main implementations of both classical and quantum machine learning models using Python and the relevant libraries. Chapter 5 will provide an in-depth analysis of the results, discussing the advantages, limitations, and potential of quantum machine learning, as well as any challenges encountered during the study. Finally, Chapter 6 will conclude the thesis by summarizing the key findings, highlighting the contributions to the field, and suggesting possible directions for future work.

Chapter 2

Background and Literature Review

2.1 Introduction

In this chapter, a background and literature review will be conducted to equip the reader with the essential theoretical foundations of Quantum Machine Learning (QML). This review will cover key concepts in classical machine learning, quantum computing, and the emerging field of Quantum Machine Learning. The goal is to provide the reader with a solid understanding of the current state of the research and the potential integrations with quantum computing and machine learning algorithms.

2.2 Machine Learning

Machine learning (ML) is a subset of AI that involves the creation of algorithms that can learn from data and enable computers to learn from experience or make decisions based on experience, just like a human. A widely accepted definition, often attributed to Arthur Samuel, is: “Machine learning is a field of study that gives computers the ability to learn without being explicitly programmed“ [9]. This field is marked by significant development across many domains since it was initiated. It is said that the pioneers of the field were Alan Turing and Arthur Samuel, among others, who proposed specific ideas forming the basis for the modern ML. These contributions include the proposal of a “learning machine“ by Turing and game-playing programs by Samuel, which provided the stepping stones to the modern ML. Machine learning enables us to tackle tasks that are too difficult to solve with fixed programs written and designed by human beings. From a scientific and philosophical point of view, machine learning is interesting because developing our understanding of it entails developing our understanding of the principles that underlie intelligence.

2.2.1 Key Concepts

There are three major machine learning concepts that are used to train a prediction model and these are: supervised, unsupervised and reinforcement learning. The supervised learning concept is when the model learns from labeled data and so the algorithm is trained on input-output value pairs. Unsupervised learning is when the model learns from unlabeled data and so the algorithm tries to find hidden patterns or structures. Last but not least, reinforcement learning is when the model learns by interacting with an environment and receiving rewards and penalties according to the choices that it makes.

Let’s dive deeper in the supervised learning as it will be the concept that this thesis will mainly explore. There are two types of supervised learning, the classification in which the output is a category or a label like if an email is a spam or a medical diagnosis if it is positive or negative and

the regression in which the output is a continuous value like predicting the house prices or stock values. There are some common machine learning algorithms in supervised learning that should be mentioned. Specifically:

1. Linear Regression:

- Purpose: Used for predicting a continuous output variable based on one or more input features.
- Mechanism: Finds the line (or hyperplane in higher dimensions) that best fits the data by minimizing the mean squared error between the actual and predicted values.

2. Logistic Regression:

- Used for binary classification problems where the output is one of two possible classes.
- Mechanism: Models the probability that an input belongs to a particular class using the logistic function.

3. Support Vector Machines (SVM):

- Purpose: Used for both classification and regression tasks.
- Mechanism: Finds the hyperplane that best separates the classes in the feature space by maximizing the margin between the classes.

4. Decision Trees:

- Purpose: Used for both classification and regression tasks.
- Mechanism: Splits the data into subsets based on feature values, creating a tree structure where each node represents a decision based on a feature.

5. Random Forests:

- Purpose: An ensemble method used for both classification and regression.
- Mechanism: Combines multiple decision trees, where each tree is trained on a different subset of the data, and the final prediction is made by averaging the predictions of all the trees (for regression) or by majority vote (for classification) [10].

2.3 Quantum Computing

Quantum computing is the field that leverages the principles of quantum mechanics to process information in fundamentally new ways. Unlike classical computing, which relies on the classical bit as the basic information unit, quantum computing uses the quantum bit or qubit which stores information in more complex forms.

2.3.1 Key Concepts

The quantum computing leverages 3 main principles of quantum mechanics:

1. **Superposition:** Is a fundamental phenomenon where a qubit can be in multiple states simultaneously or a linear combination of states often presented as

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle.$$

A noteworthy point to mention here is that α and β are complex numbers and also that $\alpha + \beta = 1$.

2. **Entanglement:** Is a fundamental phenomenon where 2 or more qubits are interconnected in such a way that the state of one qubit directly influences the state of another, no matter how far apart they are. This counterintuitive property, has famously been called by Albert Einstein as "spooky action at a distance" as there is no equivalent in classical physics.
3. **Interference:** Is a fundamental phenomenon where the probability amplitudes of different quantum states combine, leading to constructive or destructive interference, in simple words increasing or decreasing the probabilities of certain outcomes.

Another key concept of quantum computing are quantum gates that are fundamental building blocks in quantum circuits, analogous to classical logic gates. They manipulate qubits using various operations. The most famous of them are:

- **Pauli-X, Pauli-Y, Pauli-Z Gates:** These gates are analogous to the classical gate NOT but operate on the quantum world. They flip the state of a qubit across one of the basis axis (X, Y or Z). Their visual representation is:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

- **Hadamard Gate:** Creates superposition by putting qubits into an equal probability of being 0 or 1. It's visual representation is:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

- **CNOT Gate (Controlled-NOT):** Performs an XOR operation on two qubits, flipping the target qubit if the control qubit is in state 1.

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

- **Phase Shift Gate:** Also known as the S gate or P gate, it introduces a phase change without the probability of the state. It's matrix representation is:

$$S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$$

The last 3 gates (Hadamard, Phase Shift and CNOT) shape a familiar set because they can be blended to approximate every other quantum gate. With these gates, one could construct any quantum computation successfully. This set is analogous to the idea of the NAND gate in classical computing. [\[11\]](#)

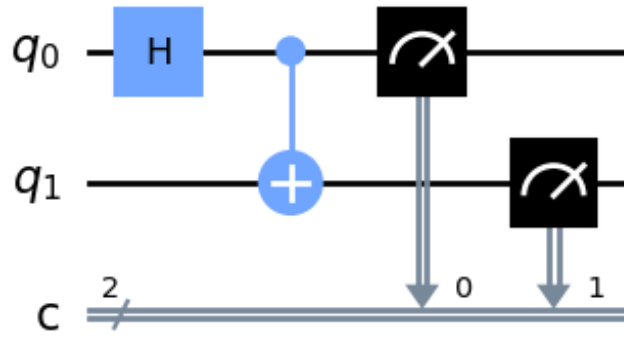


Figure 2.1: Quantum circuit implementing a simple entanglement operation with Hadamard and CNOT gates followed by measurement.

2.4 Quantum Machine Learning(QML)

Quantum Machine Learning (QML) is an emerging field that lies at the intersection of quantum computing and classical machine learning. It leverages the principles of quantum mechanics to potentially improve machine learning processes, presenting new methods to face complicated problems with more efficiency. While classical machine learning uses traditional binary computing, QML utilizes quantum bits (qubits) that can exist in multiple states simultaneously due to the phenomena of superposition and entanglement. These properties allow quantum algorithms to explore a vast solution space more efficiently than the classical algorithms under certain circumstances.

2.4.1 How Quantum Machine Learning works

Before diving into more details about the QML, it is better to understand how it actually works and what is its core differences with classical Machine Learning.

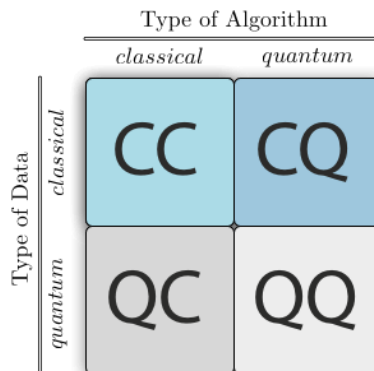


Figure 2.2: Overview of quantum machine learning approaches. Image by Schuld et al., licensed under CC BY-SA 4.0, via Wikimedia Commons (https://commons.wikimedia.org/wiki/File:Qml_approaches.tif).

What are the main areas of focus in QML? QML research is extensive and can be broadly divided into four primary sectors, each defined by the nature of the computing resources (whether the computing device is quantum ([Q]) or classical ([C])) and the type of data involved (whether generated by a quantum ([Q]) or classical ([C]) system). The explanations of these four sectors are as follows [12].

- **CC Sector:** The **CC** sector refers to classical data processed on classical systems, representing traditional machine learning. Here, classical ML algorithms run on classical processors (e.g., CPUs and GPUs) and are applied to classical datasets. A typical example is using neural networks to classify images of cats and dogs.
- **CQ Sector:** The **CQ** sector involves using classical ML algorithms on classical processors to analyze quantum data collected from quantum systems. Typical examples include applying classical neural networks to classify quantum states, estimating properties of quantum systems from measurement data, and employing classical regression models to predict outcomes of quantum experiments.
- **QC Sector:** The **QC** sector involves developing QML algorithms that run on quantum processors (QPUs) to process classical data. In this context, quantum computing resources are leveraged to enhance or accelerate the analysis of classical datasets. Typical examples include applying QML algorithms, such as quantum neural networks and quantum kernels, to improve pattern recognition in image analysis.
- **QQ Sector:** The **QQ** sector involves developing QML algorithms executed on QPUs to process quantum data. In this context, quantum computing resources are leveraged to reduce the computational cost of analyzing and understanding complex quantum systems. Typical examples include using quantum neural networks for quantum state classification and applying quantum-enhanced algorithms to simulate quantum many-body systems [12].

The primary focus of this thesis is the **CC** and **QC** sector where we will use classical data to train quantum and classical machine learning algorithms.

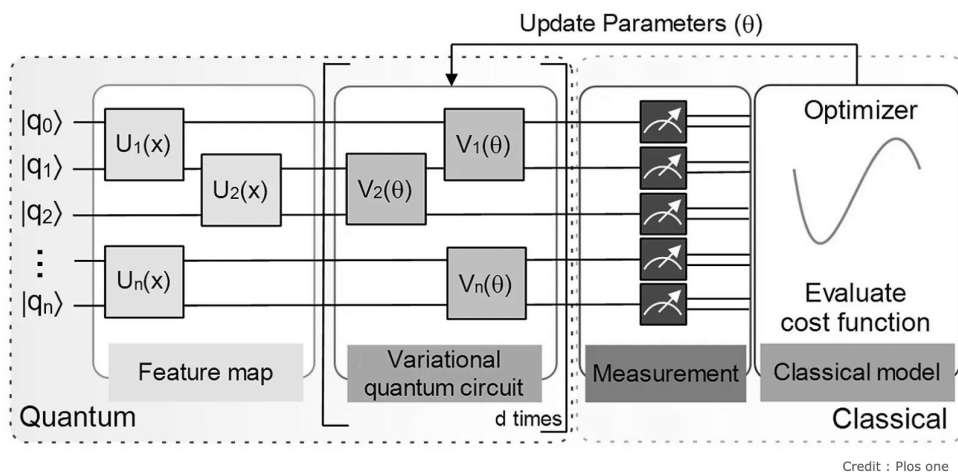


Figure 2.3: Hybrid Quantum Machine Learning circuit for Variational quantum classifiers (VQCs)

Let's use the above figure as an example to understand how a hybrid quantum machine learning algorithms works.

1. Step 1: Feature Map

We begin with a set of qubits, each initialized in a well-behaved state (ground state zero 0). The *feature map*—represented by the yellow boxes $U_1(x), U_2(x), \dots, U_n(x)$ —encodes classical data x into the quantum world. This is where your data gets its quantum passport and is ready for some serious entanglement.

2. Step 2: Variational Quantum Circuit

Next, the data passes through a series of parameterized quantum gates the blue boxes $V_1(\theta), V_2(\theta), \dots, V_n(\theta)$. These gates are controlled by parameters θ , which are the dials we tune to optimize performance. This part is repeated d times, because quantum circuits love a good remix.

3. Step 3: Measurement

After all the quantum calculations, we measure the qubits. This is where the quantum magic collapses into classical bits—like opening a mystery box and finally seeing what’s inside.

4. Step 4: Classical Optimization

Now, the classical computer takes over. It looks at the measurement results, evaluates a cost function (how well did we do?), and uses an optimizer to suggest better parameters θ for the quantum circuit. This feedback loop continues until the algorithm is happy (or at least less grumpy).

5. **In summary:** The quantum computer does the heavy lifting with data encoding and entanglement, while the classical computer acts as the coach, constantly adjusting the strategy. Together, they form a hybrid algorithm that’s greater than the sum of its parts—at least, that’s the hope!

All these steps will be explained in detail in chapter 4 of the thesis where the implementation of the Quantum Machine Learning algorithms will take place.

2.4.2 Key Algorithms in QML

There are several quantum algorithms that have potential to make a difference in comparison to the classical machine learning methods. The most notable among them are Quantum Support Vector Machines (QSVM), Quantum Neural Networks (QNN) and Quantum K-Nearest Neighbors (QKNN).

- **Quantum Support Vector Machines (QSVM):** QSVMs are an extension of classical Support Vector Machines (SVMs). They leverage the power of quantum computing to efficiently handle high-dimensional data spaces. By encoding the data into a quantum state and applying the right quantum gates on these states using a quantum circuit, QSVMs can potentially classify data more accurately and with less computational overhead than the classical ones. [13]
- **Quantum Neural Networks (QNN):** QNNs apply the principles of quantum mechanics in simulating neural networks. In a QNN the traditional layers of neurons are being replaced by quantum circuits, enabling the network to learn patterns and make predictions based on quantum states. QNNs have the potential to perform complex calculations more quickly than the classical machine learning networks, though practical implementations are still in the early stages. [6]

- **Quantum K-Nearest Neighbors (QKNN):** The QKNN algorithm uses the classical K-Nearest Neighbors method by using quantum computing to measure distances between data points in a more computationally efficient way. This can lead to faster classification, especially in scenarios involving large datasets with high-dimensional features. [14]

2.4.3 Quantum Machine Learning Libraries

As quantum computing continues to evolve, several libraries and frameworks have been developed to facilitate research and implementations in quantum machine learning which mainly are written for Python. These libraries provide high-level interfaces to simulate quantum algorithms and develop hybrid classical-quantum models where some calculations are executed in classical computers and some in quantum ones. Below is a review of the major libraries available for QML, followed by an in-depth look at the libraries used in this thesis: Qiskit and PennyLane.

Overview of Popular Existing Quantum Machine Learning Libraries

1. **Qiskit:** Qiskit is an open-source quantum computing framework developed by IBM. It provides tools for creating quantum circuits, running quantum simulations and execution on real quantum hardware. [8].
2. **PennyLane:** PennyLane is a software framework developed by Xanadu that bridges quantum computing and machine learning. It also supports hybrid quantum-classical computations and can be integrated with existing machine learning libraries like TensorFlow and PyTorch. [15].
3. **TensorFlow Quantum (TFQ):** TensorFlow Quantum developed by Google and is an extension of the TensorFlow library, designed to combine quantum computing with machine learning models. It provides tools to create hybrid quantum-classical models and is widely used for experiments with quantum circuits in machine learning [16].
4. **Cirq:** Cirq is another library from Google which is focused on developing quantum circuits and algorithms. While it doesn't include QML modules, it can be used alongside TensorFlow Quantum for implementing QML models [17].

Libraries Used

This thesis focuses on the use of Qiskit and PennyLane for implementing quantum machine learning models.

- **Qiskit**

Qiskit is one of the most widely used quantum computing libraries, designed by IBM. It allows developers to create, simulate and execute quantum circuits on both simulators and IBM's real quantum computers. The library had a major upgrade on February 2024 and is no longer a beta version, this added new components to it. Apart from the classic components of building, optimizing, executing quantum programs in simulation and real devices, now Qiskit has:

- **Qiskit Machine Learning:** A new addition to the Qiskit ecosystem, this module is dedicated to integrating quantum algorithms with machine learning tasks. Qiskit Machine Learning provides a suite of tools for building, training and testing quantum machine learning models. It allows seamless integration with classical machine learning libraries

which makes it ideal for hybrid quantum-classical workflows (which is the state of the art right now). The library also supports quantum neural networks, which are crucial in developing advanced quantum machine learning models.

- **Quantum Kernel Methods:** Qiskit new version introduced support for quantum kernel algorithms, which are useful for Support Vector Machines (SVMs) algorithms and other classification models. These kernel methods allow quantum computers to process higher-dimensional data more effectively than classical machines.
- **Improved Noise Models:** Noise simulation in Qiskit has been made more realistic, accounting for new quantum errors encountered in IBM's latest hardware. This ensures that simulations more closely resemble real quantum hardware execution, providing more accurate results when developing quantum algorithms.
- **Advanced Optimization for Circuits:** The latest version also includes new tools for optimizing quantum circuits. Users can now apply transpiler-level optimizations, which can reduce the number of gates in circuits and minimize the probability of errors during the execution [8].

- **PennyLane**

PennyLane is a cross-platform quantum machine learning library developed by Xanadu, which allows for the integration of quantum computing with classical machine learning models. It is particularly designed for hybrid quantum-classical systems, which is crucial for quantum machine learning. Some of PennyLane's key features include:

- **Hybrid Quantum-Classical Models:** PennyLane allows for the seamless combination of quantum and classical layers, facilitating the integration of quantum circuits with classical neural networks.
- **Differentiable Programming:** One of the key advantages of PennyLane is its support for automatic differentiation, making it easier to optimize quantum circuits using gradient-based methods.
- **Quantum Gradients:** PennyLane support quantum gradients, allowing users to efficiently compute the gradients of quantum circuits during the optimization process.

In this research, PennyLane was used for implementing hybrid quantum-classical machine learning models, where the quantum circuit acted as a feature extractor and was combined with a classical neural network. [15]

2.4.4 Advantages and Challenges of QML

Quantum Machine Learning offers many potential advantages over classical approaches such as:

- **Increased Computational Efficiency:** Quantum algorithms can process information faster than the classical ones for certain tasks, such as factorization, search and optimization. [11]
- **Handling Complex Data Structures:** Quantum Machine Learning has the potential to manage and analyze complex, high-dimensional datasets that are challenging for classical algorithms to process. [6]

However, Quantum Machine Learning also faces significant challenges such as:

- **Technological Limitations:** The development of quantum computers are still in its early days, with the current hardware being prone to high error rates and with the small amount of qubits that are available in a quantum processor, making it difficult to scale up quantum algorithms for practical implementations. [18]
- **Algorithmic Complexity:** Designing quantum algorithms that outperform classical ones is not an easy task, it requires a deep understanding of both quantum mechanics and machine learning. [19]

2.4.5 Recent Developments and Applications

The field of Quantum Machine Learning has witness many promising developments and applications over the last years that highlight its potential across various areas. Some notable examples are:

- **Quantum Algorithms for Financial Markets:** Researchers are exploring the use of quantum machine learning for option pricing and risk analysis. Specifically, they made some experiments with quantum algorithms for Monte Carlo simulations that have demonstrated a potential speed up in complex financial calculations that are critical for pricing derivatives and assessing risk. [20]
- **Drug Discovery and Molecular Modeling:** Researchers are using Quantum Machine Learning to simulate molecular interactions at unprecedented speeds, which potentially can reduce the time and cost associated with discovering new drugs. An example of that is the company D-Wave that have collaborated with biotech firms to use quantum computing for the protein folding and gene analysis, which are complex problems that require immense computational resources. [21]
- **Quantum Machine Learning in Cybersecurity:** The recent year there is an increase in the data breaches and cyber threads rate and quantum computing offers some new methodologies for cybersecurity that can potentially make computer safer. For example Quantum key distribution (QKD) and quantum random number generator (QRNG) are technologies that could be leverage encryption and provide a superior security compared to classical methods [22]

2.5 Theoretical Comparison Between Classical and Quantum Machine Learning

With a more clear understanding of both classical machine learning and quantum machine learning, it is vital to compare these two problem solving approaches and understand their strengths and weaknesses.

- **Performance and Efficiency:** Even though classical machine learning algorithms have been optimized over decades, making them highly efficient for a wide range of applications, the quantum machine learning algorithms, by leveraging quantum parallelism, promise to solve certain problems much faster, particularly those involving large and complex datasets [11]. But in order for the quantum algorithms to make the difference here, researchers have to wait for the better hardware to be developed.
- **Scalability:** Classical machine learning algorithms often struggle with scalability when applied to extremely large datasets. Quantum algorithms, on the other hand, could offer better

scalability by processing vast volumes of data in parallel, especially within high-dimensional spaces [13].

- **Error Tolerance and Robustness:** Quantum systems are currently more error-prone than classical systems due to decoherence and noise in quantum circuits. This makes quantum algorithms less robust in their current form, though ongoing research aims to mitigate these issues [18].
- **Case Studies:** Various studies have compared the performance of classical and quantum algorithms on tasks like classification and clustering. These comparisons often highlight the potential of quantum algorithms to outperform classical ones as quantum hardware continues to improve [6].

This is also the goal of this thesis, to compare the performance of classical and quantum machine learning on classification problems and explore any potential improvements and insight.

2.5.1 Model-Level Analogies Between Classical and Quantum ML

Beyond high-level pros and cons, many classical models have clear quantum counterparts. These analogies help demystify what “goes quantum” while keeping the learning principles familiar.

1. Linear/Kernel Methods $\leftrightarrow$ Quantum Kernel Methods

Classical: Support Vector Machines (SVMs) and kernel methods compute similarities via an inner product in a (possibly implicit) feature space, $K(x, x') = \langle \phi(x), \phi(x') \rangle$.

Quantum: Quantum kernels embed data with a feature map $U_\phi(x)$ into a quantum state $|\phi(x)\rangle = U_\phi(x)|0 \cdots 0\rangle$ and measure similarity via state fidelity,

$$K_Q(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2,$$

estimated with circuits such as the Hadamard or SWAP test [6, 11, 13]. Intuitively, both worlds exploit geometry induced by an inner product, quantum circuits define that geometry in a high-dimensional Hilbert space that may be hard to emulate classically.

2. Logistic Regression / Linear Classifiers $\leftrightarrow$ Shallow VQC with Single-Qubit Readout

Classical: Logistic regression models $\sigma(w^\top x + b)$ and optimizes cross-entropy.

Quantum: A variational quantum classifier (VQC) with data encoding $U_\phi(x)$, a shallow ansatz U_θ , and a single Z -measurement yields an expectation $\langle Z \rangle_\theta(x)$, which after an affine map and sigmoid plays the same role as a logit. Training minimizes the same loss, using parameter-shift or gradient-free optimizers [6, 13].

3. Neural Networks (ANNs) $\leftrightarrow$ Quantum Neural Networks (QNNs)

Classical: Stacks of affine layers and nonlinearities; depth increases expressivity but complicates optimization.

Quantum: Alternating data re-uploading and parametrized layers with entanglers. Nonlinearity emerges from measurement statistics and trigonometric parameterization. Depth and entanglement increase expressivity, but can induce barren plateaus (vanishing gradients) on random initializations in NISQ regimes [13, 18].

Table 2.1: Side-by-side analogy between classical and quantum ML models and mechanisms.

Aspect	Classical ML	Quantum ML
Kernel methods	SVM with $K(x, x')$ (e.g., RBF)	QSVM with kernel from circuit overlaps (Hadamard/SWAP test)
Linear classifier	Logistic regression $\sigma(w^\top x + b)$	Shallow VQC: $\sigma(\alpha \langle Z \rangle_\theta(x) + \beta)$
Deep models	ANN/CNN layers + nonlinearities	QNN layers (parametrized rotations + entanglers) + measurement

The quantum column emphasizes state-fidelity kernels and parametrized circuits while the classical column emphasizes vector-space inner products and gradient backprop. Both rely on induced geometries for generalization.

Chapter 3

Methodology - My Quantum (and Classical) Quest

In this chapter, I'll lay out the battle plan for my epic showdown between classical and quantum machine learning. Think of it as the pre-fight weigh-in, where we size up the contenders and lay down the rules of engagement. We're going to be looking at how these algorithms perform on real-world(ish) datasets, using both the tried-and-true methods of classical machine learning and the flashy, still-somewhat-mysterious techniques of quantum machine learning.

3.1 Datasets Description: The Training Grounds (A Deeper Dive)

Alright, let's talk data! Before we can have our epic clash of classical and quantum algorithms, we need something for them to learn from. Think of these datasets as the training grounds for our digital gladiators, where they'll hone their skills before the main event. I've chosen three datasets from the vast and sometimes wild world of Kaggle, both related to healthcare – because what's more important than keeping people healthy (and because healthcare data is often nicely structured for machine learning!). These aren't just random collections of numbers; they represent real-world challenges, and they're a good size for our purposes – big enough to be meaningful, but not so huge that they'll overwhelm our (especially quantum) computational resources.

3.1.1 Heart Failure Prediction Dataset: The Heart of the Matter

This dataset is like a snapshot of a patient's cardiovascular health. Imagine a doctor's detailed notes, neatly organized into a spreadsheet – that's essentially what we have here. We've got 918 patient records, each one a unique individual with their own medical story. And for each patient, we have 12 features, which are basically different pieces of information about them. These features include:

- **Demographics:** Age (because time marches on, even in datasets), Sex (Male/Female).
- **Clinical Measurements:** Cholesterol levels (the good, the bad, and the ugly), Resting Blood Pressure (hopefully not too high!), Fasting Blood Sugar (a sweet indicator).
- **Lifestyle Factors:** Exercise Angina (chest pain during exercise - not a good sign), ST Slope (a measure of heart activity during exercise).
- **Chest Pain Type:** Categorical, because not all chest pains are created equal.
- **Resting ECG:** Another measure of heart activity, this time at rest.

- **MaxHR**: Maximum heart rate achieved.
- **Oldpeak**: ST depression induced by exercise relative to rest.

Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0

Table 3.1: Snapshot of the Heart Failure Prediction Dataset

The target variable **HeartDisease** – the thing we’re trying to predict – is whether the patient has heart disease (1) or not (0). It’s a classic binary classification problem: yes or no, heart disease or no heart disease. Now, here’s where the data wrangling comes in. We’ve got a mix of numerical features (like age and cholesterol, which are numbers) and categorical features (like chest pain type, which is a category – “typical angina,” “atypical angina,” etc.). Computers, especially quantum computers, prefer numbers. So, we need to do some preprocessing:

- **Categorical Encoding**: Features like “Sex” and “ChestPainType” were label-encoded. This means turning words into numbers (e.g., “Male” becomes 0, “Female” becomes 1). The `LabelEncoder` from `scikit-learn` was our trusty tool for this.
- **Numerical Scaling**: We used Min-Max scaling to squish all numerical features between 0 and 1. This prevents features with large values (like cholesterol) from unfairly dominating the models. It’s like giving all the features an equal playing field.

Why this dataset? Well, heart disease is a big deal. It’s a leading cause of death worldwide. Being able to predict it accurately could help doctors intervene early and save lives. Plus, the mix of numerical and categorical features makes it a good test case for our machine learning models, both classical and quantum. It’s a challenging but manageable problem with only 918 rows for training and validation let’s see how good the classical machine learning models will perform.

3.1.2 Stroke Prediction Dataset: Brain Games

This one deserves a bit more attention. We’re shifting our focus from the heart to the brain, and the stakes are just as high. Strokes are a leading cause of disability and death, and being able to predict them could be a game-changer. The dataset contains 5,110 patient records, more data than the heart dataset, which is always a good thing (usually!). More data means more opportunities for our algorithms to learn, but also more computational resources required. Also, the dataset has 11 features, a slightly leaner feature set than the heart dataset, but still a good mix:

- **Demographics**: Age (because time marches on, even in datasets), Gender (Male/Female).
- **Medical History**: Hypertension (yes/no), heart disease (yes/no), ever married (yes/no).
- **Lifestyle Factors**: Smoking status (never smoked, formerly smoked, smokes, unknown). Work type (Private, Self-employed, Govt_job, children, Never_worked).
- **Biometric data**: Average glucose level and BMI.

ID	Gender	Age	Hypertension	HeartDisease	Married	Work	Residence	AvgGlucose	BMI	Smoke	Stroke
9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Table 3.2: Snapshot of the Stroke Prediction Dataset

The target variable is **Stroke** which takes the values (0) for no stroke and (1) for stroke and yet it's another binary classification problem. The preprocessing consist of the same drill as the heart dataset: label encoding for the categorical features (and there are quite a few!), and min-max scaling for the numerical ones.

Another major characteristic of this dataset is it's imbalance issue. Specifically the Stroke Prediciton dataset is highly imbalanced. This means that there are far fewer instances of strokes (1s) than non-strokes (0s). This is a common problem in real-world datasets, especially in health-care, where the event you're trying to predict is (thankfully!) relatively rare. An imbalanced dataset can seriously mess with our machine learning models. Imagine a model that always predicts "no stroke" – it would be right most of the time, but completely useless! We'll need to address this imbalance during our preprocessing and model evaluation, using techniques like oversampling, undersampling, or adjusting class weights. This is a perfect example of the "power of data" challenge – even with a lot of data, its distribution matters immensely.

3.1.3 The Big Picture

With these two datasets (Heart Failure and Stroke dataset), we have a robust and diverse set of healthcare-related challenges. We have different dataset sizes, a mix of feature types, and the important issue of class imbalance. This allows for a thorough investigation of QML's strengths and weaknesses within a focused domain, addressing key challenges like input/output handling, computational cost, benchmarking against classical methods, and the crucial role of data characteristics.

3.2 Classical Machine Learning Models: Our Baseline Arsenal

To establish a rock-solid baseline for our quantum showdown, we've assembled a team of classical machine learning all-stars. These aren't just any algorithms, they're the tried-and-true workhorses of the classification world, implemented with the precision and efficiency of the scikit-learn library in Python. This ensures our comparison is fair, reproducible, and focused on the algorithmic differences, not implementation quirks. We're covering a range of approaches, from the elegantly simple to the powerfully complex, to make sure our quantum models have a real challenge.

Here's the technical breakdown of our classical contenders:

3.2.1 Logistic Regression: The Linear Linebacker

- **The Math:** Logistic Regression models the probability of a binary outcome (0 or 1) using the logistic (sigmoid) function:

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

where:

- $\mathbf{x}$ is the input feature vector.
 - $\mathbf{w}$ is the weight vector (the parameters we learn).
 - b is the bias term.
 - $\sigma(z)$ is the sigmoid function, squashing the output to be between 0 and 1 (representing a probability).
- **The Algorithm:**
 - **Initialization:** Initialize the weights $\mathbf{w}$ and bias b (often randomly or to zero).
 - **Prediction:** For a given input $\mathbf{x}$, calculate the predicted probability $P(y = 1|\mathbf{x})$.
 - **Loss Function:** We use the log loss (also called cross-entropy loss) to measure how well our predictions match the true labels:

$$L(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(P(y_i = 1|\mathbf{x}_i)) + (1 - y_i) \log(1 - P(y_i = 1|\mathbf{x}_i))]$$

where N is the number of training samples, and y_i is the true label (0 or 1) for the i -th sample [10].

- **Optimization:** We use gradient descent (or a variant like stochastic gradient descent, Adam, etc.) to minimize the loss function. This involves calculating the loss gradients with respect to $\mathbf{w}$ and b , and updating the parameters iteratively:
$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial L}{\partial \mathbf{w}}$$
$$b \leftarrow b - \eta \frac{\partial L}{\partial b}$$
- where η is the learning rate.
- **Classification:** Once trained, we classify a new input $\mathbf{x}$ by thresholding the predicted probability: if $P(y=1 | \mathbf{x}) \geq 0.5$, predict class 1; otherwise, predict class 0.
- **Why it's here:** Provides a simple, interpretable, linear baseline. It's the "control group" of our experiment.
 - **Strengths:** Fast, easy to understand, works well for linearly separable data, provides probabilistic outputs.
 - **Weaknesses:** Can't model complex non-linear relationships without feature engineering.

- **Plot:**

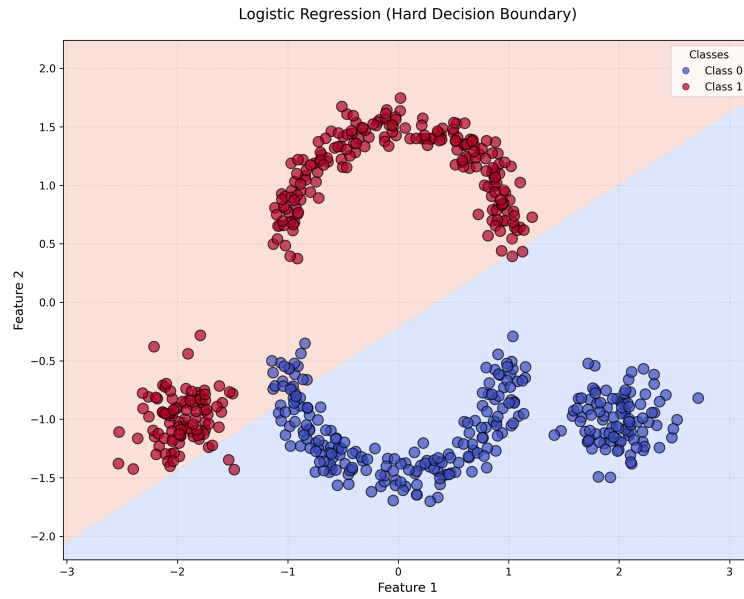


Figure 3.1: Simple Logistic Regression implementation

3.2.2 Naive Bayes: The Probabilistic Powerhouse (with a Big Assumption)

- **The Math:** Naive Bayes is based on Bayes' theorem:

$$P(y|\mathbf{x}) = \frac{P(\mathbf{x}|y)P(y)}{P(\mathbf{x})}$$

The "naive" part is the assumption that features are conditionally independent given the class label:

$$P(\mathbf{x}|y) = \prod_{j=1}^d P(x_j|y)$$

where d is the number of features. This simplifies the calculations dramatically.

- **The Algorithm:** We'll focus on Gaussian Naive Bayes, which assumes that the likelihood of the features given the class is Gaussian:

$$P(x_j|y) = \frac{1}{\sqrt{2\pi\sigma_{yj}^2}} \exp\left(-\frac{(x_j - \mu_{yj})^2}{2\sigma_{yj}^2}\right)$$

where:

- μ_{yj} is the mean of feature j for class y.
- σ_{yj}^2 is the variance of feature j for class y.
- **Training:** For each class y, calculate the prior probability $P(y)$ (the proportion of training samples belonging to class y). For each feature j and class y, calculate the mean μ_{yj} and variance σ_{yj}^2 from the training data [10].

- **Prediction:** For a new input x , calculate $P(y|x)$ for each class y using Bayes' theorem and the conditional independence assumption. Since $P(x)$ is the same for all classes, we can just compare the numerators:

$$\hat{y} = \arg \max_y P(y) \prod_{j=1}^d P(x_j|y)$$

(In practice, we often work with log-probabilities to avoid numerical underflow.)

- **Classification:** Assign x to the class with the highest (log-)probability.
- **Why it's here:** Computationally efficient, works well with high-dimensional data, even with the strong independence assumption.
- **Strengths:** Fast, simple, good for high-dimensional data (like text data), surprisingly effective.
- **Weaknesses:** The feature independence assumption is often violated, which can limit accuracy.
- **Plot:**

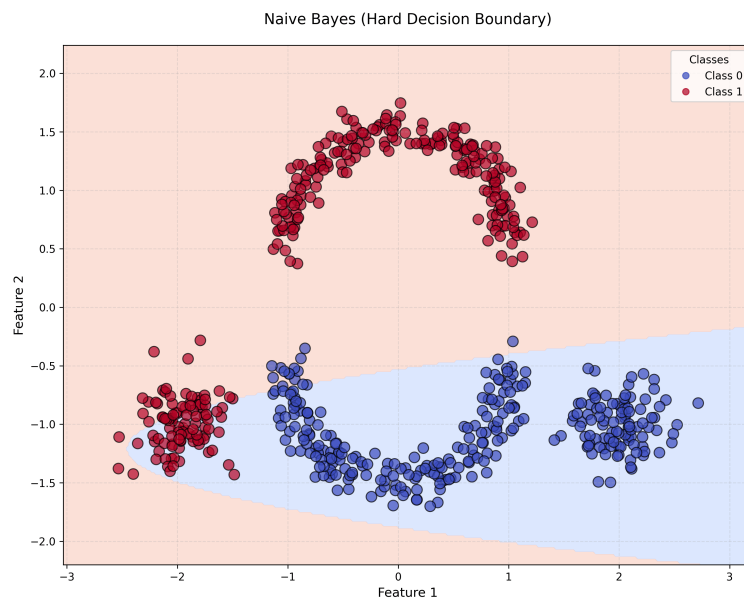


Figure 3.2: Simple Naive Bayes implementation

3.2.3 Support Vector Machines (SVM) - Linear Kernel: The Margin Maximizer

- **The Math:** SVMs find the hyperplane that maximizes the margin between the classes. For a linearly separable dataset, the decision boundary is:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

The "naive" part is the assumption that features are conditionally independent given the class label:

$$P(\mathbf{x}|y) = \prod_{j=1}^d P(x_j|y)$$

The goal is to find w and b that maximize the margin while correctly classifying the training data. This is formulated as a quadratic programming problem: Minimize:

$$\frac{1}{2} ||w||^2$$

Subject to:

$$y_i(w^T x_i + b) \geq 1$$

for all training samples x_i, y_i . The y_i values are +1 or -1 (not 0 and 1, as in Logistic Regression).

- **The Algorithm:** SVM training is typically done using specialized optimization solvers (like libsvm or liblinear) that efficiently solve the quadratic programming problem. There's no simple iterative update like gradient descent. The solver finds the optimal w and b [10].
- **Prediction:** For a new input x , the classification is:

$$\hat{y} = \text{sign}(w^T x + b)$$

If the result is positive, predict class +1; if negative, predict class -1.

- **Why it's here:** Effective for linearly separable data, computationally efficient (with the linear kernel).
- **Strengths:** Good for linearly separable data, efficient, robust to outliers (with appropriate regularization).
- **Weaknesses:** Can't model non-linear relationships (without a different kernel).

- **Plot:**

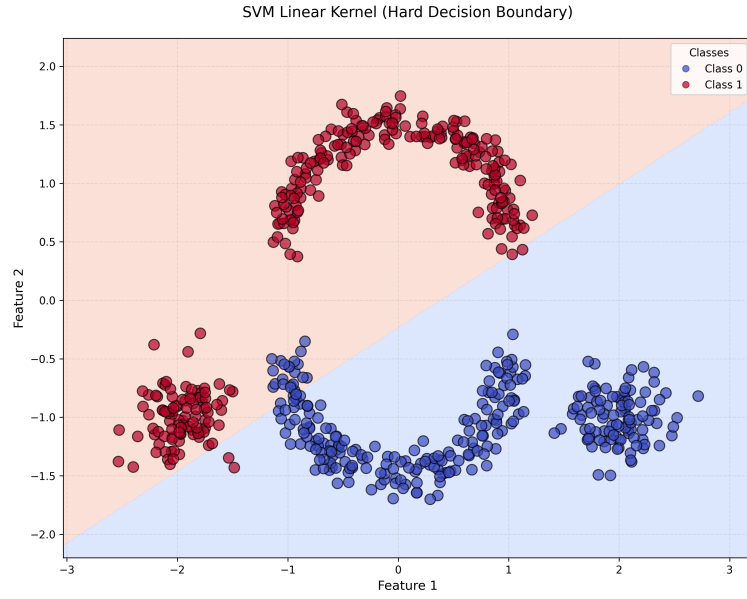


Figure 3.3: Simple Support Vector Machine (Linear) implementation

3.2.4 Support Vector Machines (SVM) - RBF Kernel: The Non-Linear Navigator

- **The Math:** The RBF (Radial Basis Function) kernel allows SVMs to model non-linear decision boundaries. The kernel function is:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

where γ (gamma) is a hyperparameter that controls the width of the Gaussian. The kernel implicitly maps the data into a higher-dimensional feature space where the data might be linearly separable.

- **The Algorithm:** Similar to the linear SVM, we use a solver to find the optimal parameters. However, instead of directly working with w and b , we work with dual variables (α_i) associated with each training sample. The optimization problem becomes: Minimize:

$$\text{Minimize: } \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) - \sum_{i=1}^N \alpha_i$$

Subject to:

$$0 \leq \alpha_i \leq C \text{ for all } i, \text{ and } \sum_{i=1}^N \alpha_i y_i = 0.$$

C is a regularization parameter (controlling the trade-off between margin maximization and classification errors) [10].

- **Prediction:** For a new input $\mathbf{x}$, the classification is:

$$\hat{y} = \text{sign} \left(\sum_{i=1}^N \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b \right)$$

Note that we only need to compute the kernel function between the new input $\mathbf{x}$ and the support vectors (the training samples with non-zero α_i).

- **Why it's here:** Can model complex non-linear relationships.
- **Strengths:** Powerful, flexible, can model complex data.
- **Weaknesses:** More computationally expensive than linear SVM, requires tuning γ and C .
- **Plot:**

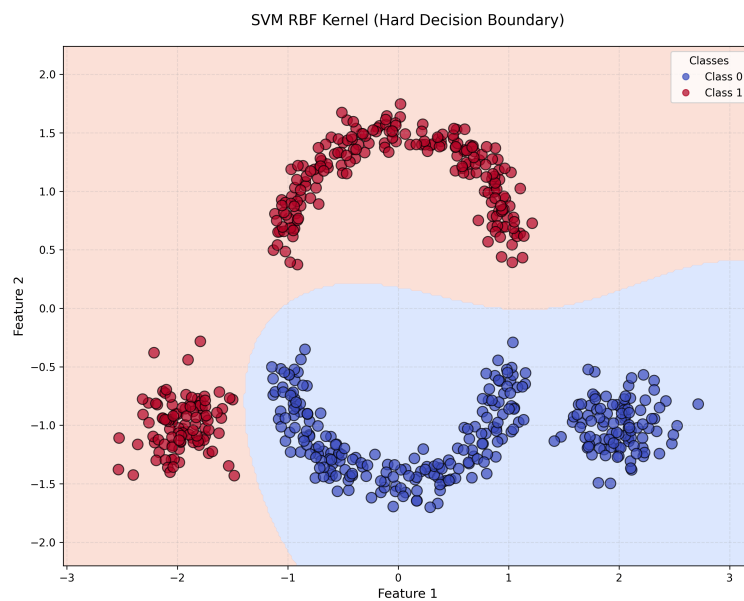


Figure 3.4: Simple Support Vector Machine (RBF) implementation

3.2.5 K-Nearest Neighbors (KNN): The Neighborhood Knower

- **The Math:** KNN is a non-parametric method. It doesn't learn a model in the traditional sense. Instead, it stores the training data. The key is the distance metric (usually Euclidean distance):

$$d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\| = \sqrt{\sum_{k=1}^d (x_{ik} - x_{jk})^2}$$

where γ (gamma) is a hyperparameter that controls the width of the Gaussian. The kernel implicitly maps the data into a higher-dimensional feature space where the data might be linearly separable [10].

- **The Algorithm:** Store all training data points $(\mathbf{x}_i, y_i)$.
- **Prediction:** For a new input $\mathbf{x}$:

1. Calculate the distance $d(x, x_i)$ between x and every training point x_i .
 2. Find the k training points closest to x (the k nearest neighbors).
 3. Assign x the majority class among its k nearest neighbors.
- **Why it's here:** Simple, non-parametric, can model complex boundaries.
 - **Strengths:** Easy to implement, no training phase (lazy learner).
 - **Weaknesses:** Can be slow for large datasets (needs to compute distances to all training points), sensitive to the choice of k and the distance metric, sensitive to irrelevant features.
 - **Plot:**

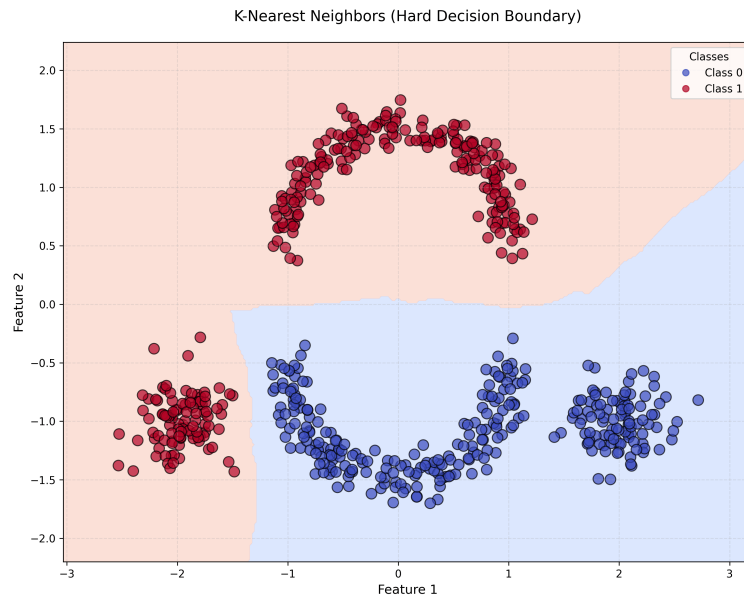


Figure 3.5: Simple K Nearest Neighbors implementation

This selection provides a strong, diverse baseline. We have linear (Logistic Regression, Linear SVM) and non-linear (RBF SVM, KNN) models, probabilistic (Naive Bayes, Logistic Regression) and geometric (SVM) approaches, and instance-based learning (KNN). We're using scikit-learn for consistency and reproducibility. While other powerful models exist (Random Forests, GBMs), this set gives us a deep understanding of the classical landscape before we introduce the quantum contenders. This focused approach allows for a more meaningful comparison, highlighting the algorithmic differences between classical and quantum machine learning.

3.3 Quantum Machine Learning Models: Entering the Quantum Realm

Having established a firm foundation with our classical models, we now venture into the exciting and often perplexing world of quantum machine learning. We've implemented three distinct quantum algorithms, each leveraging quantum principles in different ways to tackle our classification problems. These models, built using IBM's quantum computing library Qiskit, represent a spectrum of approaches, from variational methods to kernel-based techniques and hybrid quantum-classical neural networks.

3.3.1 Variational Quantum Classifier (VQC): The Quantum Gymnast

The Variational Quantum Classifier (VQC) is a prime example of a variational quantum algorithm. It's like a quantum gymnast, adjusting its "pose" (the quantum circuit) to best fit the data. The VQC consists of two main components: a quantum feature map and a variational quantum circuit (ansatz).

- **Quantum Feature Map:** This is where we encode our classical data into a quantum state. We're using a ZZFeatureMap, which creates entanglement between qubits based on the input features. It's like giving each data point a unique quantum "fingerprint." The ZZFeatureMap encodes the classical features into the rotation angles of the qubits, creating entanglement based on pairwise feature interactions. The encoding is defined as:

$$|x\rangle = U_{\text{FM}}(x)|0\rangle^{\otimes n}$$

where $|0\rangle^{\otimes n}$ is the initial state of the n qubits, and $U_{\text{FM}}(x)$ is the feature map unitary, parameterized by the classical data x . The ZZFeatureMap applies rotations based on the following:

$$U_{\text{ZZ}}(x) = \exp\left(i \sum_{i < j} x_i x_j Z_i Z_j\right)$$

where Z_i and Z_j are Pauli-Z operators on qubits i and j , and x_i and x_j are the corresponding features [8][12].

- **Variational Quantum Circuit (Ansatz):** This is a parameterized quantum circuit whose parameters we optimize during training. We're using a TwoLocal ansatz, which consists of alternating layers of single-qubit rotations (Ry gates) and CNOT gates. The Ry gates are parameterized, meaning their rotation angles are adjusted by the classical optimizer. The CNOT gates create entanglement, allowing the circuit to learn complex relationships between the qubits. It's like a series of quantum "muscles" that we train to perform the classification task. Our implementation utilizes a TwoLocal ansatz, consisting of alternating layers of single-qubit rotations (Ry gates) and CNOT gates. The choice of the TwoLocal ansatz was motivated by its balance between expressibility and gate count, allowing us to explore a reasonably complex hypothesis space while remaining within the constraints of near-term quantum devices. The single-qubit rotations are parameterized as:

$$U(\theta) = \exp(-i\theta\sigma/2)$$

where θ is the rotation angle and σ is a Pauli operator (in this case, σ_y). The CNOT gates introduce entanglement, enabling the circuit to learn correlations between qubits. The specific arrangement of R_y and CNOT gates within the TwoLocal ansatz is crucial for its ability to represent complex decision boundaries [8].

- **Optimizer:** We train the VQC by iteratively adjusting the parameters of the variational quantum circuit to minimize a cost function. We're using the COBYLA optimizer, a gradient-free optimization algorithm that's well-suited for noisy quantum circuits. It's like a coach guiding the quantum gymnast to find the perfect routine. The COBYLA optimizer iteratively adjusts the parameters based on function evaluations, without requiring explicit gradient calculations.
- **Measurement:** To get a classification result, we measure the expectation value of the PauliZ operator on the last qubit. The sign of the expectation value determines the predicted class. It's like reading the gymnast's score after the performance.

- **Why it's here:** The VQC is a flexible and widely-used quantum classification algorithm that can be implemented on near-term quantum computers. It allows us to explore the potential of variational quantum algorithms for machine learning.

3.3.2 QSVM Pegasos: The Quantum Kernel Crusader

The Quantum Support Vector Machine (QSVM) leverages the power of quantum computers to calculate kernel matrices, which are then used in a classical SVM algorithm. We're using a variant called Pegasos, which is a stochastic gradient descent algorithm for training SVMs. It's like a quantum-powered knight, wielding a kernel matrix to defend the boundaries between classes.

- **Quantum Kernel:** The heart of the QSVM is the quantum kernel, which measures the similarity between data points in a quantum feature space. We're using a kernel based on the PauliFeatureMap, which encodes the data into a quantum state using Pauli operators. The kernel value is estimated using a quantum circuit that implements a Hadamard test. It's like a quantum "similarity detector" that reveals hidden relationships between data points. The quantum kernel is constructed using a PauliFeatureMap feature map. The kernel value between two data points x and y is estimated using a quantum circuit that implements the following:

$$K(x, y) = |\langle \phi(x) | \phi(y) \rangle|^2$$

where $|\phi(x)\rangle$ and $|\phi(y)\rangle$ are the quantum states corresponding to the classical data points x and y , and the inner product is estimated using a Hadamard test. The Hadamard test involves creating a superposition state, applying controlled versions of the feature map unitaries, and measuring the resulting state to estimate the real part of the inner product [8].

- **Pegasos Algorithm:** We use the Pegasos algorithm to train the SVM using the quantum-calculated kernel matrix. Pegasos is an iterative algorithm that updates the SVM weights based on the training data. It's like a skilled swordsman, adjusting the blade to strike the perfect blow. The Pegasos algorithm iteratively updates the SVM weights based on the training data and the quantum-calculated kernel matrix. The update rule is:

$$w_{t+1} = (1 - \eta_t \lambda) w_t + \eta_t y_t \phi(x_t)$$

where w_t is the weight vector at iteration t , η_t is the learning rate, λ is the regularization parameter, y_t is the label of the t -th training sample, and $\phi(x_t)$ is the quantum feature map of the t -th training sample. The regularization parameter λ controls the trade-off between maximizing the margin and minimizing the classification error. The choice of λ is crucial for achieving good generalization performance.

- **Why it's here:** The QSVM demonstrates how quantum computers can be used to enhance classical machine learning algorithms by providing more powerful kernel functions.

To clarify, it's important to distinguish between the standard Quantum Support Vector Machine (QSVM) algorithm and the Pegasos variant we've implemented. The standard QSVM, as initially proposed, often relies on calculating the full kernel matrix using quantum circuits and then feeding this matrix into a classical SVM solver. This approach can be computationally expensive, especially for large datasets, as it requires calculating and storing all pairwise kernel values.

Pegasos, on the other hand, offers a more efficient alternative. Pegasos (Primal Estimated sub-GrAdient Solver for SVM) is an iterative algorithm that directly optimizes the SVM objective

function using stochastic gradient descent. Instead of calculating the entire kernel matrix, Pegasos only requires estimating the kernel values for a small subset of data points in each iteration. This makes it particularly well-suited for quantum implementations, as it reduces the number of quantum circuit evaluations needed during training.

The key advantages of using Pegasos QSVM over the standard QSVM are:

1. **Reduced Quantum Resource Requirements:** Pegasos avoids the need to calculate and store the entire kernel matrix, significantly reducing the quantum resources (qubits and circuit depth) required for training. This is crucial for implementations on near-term quantum devices with limited qubit counts and coherence times.
2. **Improved Scalability:** The iterative nature of Pegasos allows it to scale more effectively to larger datasets compared to the standard QSVM, which can become computationally intractable for large kernel matrices.
3. **Suitability for Noisy Environments:** The stochastic gradient descent approach used in Pegasos can be more robust to noise in the quantum circuit evaluations compared to methods that rely on precise kernel matrix calculations.

We chose to implement the Pegasos QSVM due to these advantages, particularly its reduced quantum resource requirements and improved scalability. Given the limitations of current quantum hardware, the Pegasos variant offers a more practical and feasible approach to exploring the potential of quantum SVMs for our classification tasks.

3.3.3 Sampler/Estimator QNN: The Hybrid Harmony

The Sampler/Estimator QNN is a hybrid quantum-classical neural network that combines quantum layers with classical readout layers. We're using the EstimatorQNN class from Qiskit, which uses the Estimator primitive to calculate expectation values. It's like a harmonious blend of quantum and classical instruments, creating a unique and powerful melody.

- **Quantum Layers:** The QNN consists of a sequence of quantum layers, each of which applies a parameterized quantum circuit to the input state. These layers are designed to extract relevant features from the quantum data.
- **Classical Readout Layer:** The output of the quantum layers is then fed into a classical readout layer, which performs the final classification. We're using a sigmoid layer, which outputs a probability between 0 and 1. The sigmoid layer is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

where x is the input from the quantum layers.

- **Training:** We train the QNN using a classical optimizer (Adam) and a binary cross-entropy loss function. The optimizer adjusts the parameters of both the quantum and classical layers to minimize the loss. The binary cross-entropy loss is defined as:

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

where y is the true label and $\hat{y}$ is the predicted probability.

- **Why it's here:** The Sampler/Estimator QNN showcases the potential of hybrid quantum-classical neural networks for machine learning. It allows us to combine the strengths of both quantum and classical computing to achieve better performance [8][12].

3.3.4 Evaluation Metrics: Judging the Quantum Jockeys

Now that we've assembled our team of classical and quantum machine learning models, it's time to put them to the test! But how do we know which model is the true champion? That's where evaluation metrics come in. These metrics provide a quantitative way to assess the performance of each model and compare them fairly. We'll be using a few key metrics to judge our quantum jockeys:

- **Accuracy: The Straight-A Student** Accuracy is the most straightforward metric: it measures the overall correctness of the model's predictions. It's simply the ratio of correctly classified instances to the total number of instances.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

While accuracy is easy to understand, it can be misleading when dealing with imbalanced datasets (where one class has significantly more instances than the other). Imagine a model that always predicts "no stroke" in a dataset where only 5% of patients had a stroke. It would achieve 95% accuracy, but it would be completely useless for identifying stroke patients!

- **Precision: The Eagle-Eyed Spotter** Precision measures the model's ability to correctly identify positive instances (e.g., patients with heart disease) out of all the instances it predicted as positive. It's like an eagle-eyed spotter, focusing on the accuracy of positive predictions.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

High precision means that when the model predicts a positive outcome, it's likely to be correct. This is particularly important in medical applications where false positives can lead to unnecessary treatments or anxiety [23].

- **Recall: The Net That Catches Everything** Recall (also known as sensitivity) measures the model's ability to identify all the positive instances in the dataset. It's like a net that catches everything, ensuring that no positive instance is missed.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

High recall means that the model is good at finding all the positive cases, even if it means making some false positive predictions along the way. This is crucial when missing a positive case has serious consequences (e.g., failing to diagnose a life-threatening condition).

- **F1-Score: The Harmonious Balance** The F1-score is the harmonic mean of precision and recall. It provides a balanced measure of the model's performance, taking into account both its ability to avoid false positives and its ability to avoid false negatives [23].

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1-score is particularly useful when you want to find a model that performs well on both precision and recall, especially when the classes are imbalanced [23].

- **Area Under the Receiver Operating Characteristic Curve (AUC-ROC)** The AUC-ROC curve provides a comprehensive view of the model's performance across different classification thresholds. It plots the true positive rate (recall) against the false positive rate for various threshold settings. The AUC value represents the area under this curve, with a higher AUC indicating better performance. A perfect classifier has an AUC of 1.0, while a random classifier has an AUC of 0.5. The AUC-ROC curve is particularly useful for comparing models and understanding their trade-offs between true positives and false positives [23].

We've chosen these metrics because they provide a well-rounded assessment of our models' performance. Accuracy gives us a general sense of correctness, while precision and recall help us understand the model's behavior with respect to positive and negative instances. The F1-score provides a balanced measure, and the AUC-ROC curve gives us a comprehensive view of the model's performance across different thresholds.

By carefully analyzing these metrics, we can gain valuable insights into the strengths and weaknesses of each model and determine which ones are best suited for our classification tasks.

Chapter 4

Code Implementation- Setting Up The Arena

This chapter is the garage where we manufacture the fighter for the classical versus quantum machine learning match, and we transition from theory into action. We translate the concepts discussed in the last chapter into concrete code, because we need to build the arena where our battle will occur. We will have neat pipelines to prepare our data, so we will optimize our models, and we will sprinkle them with quantum. From uninteresting, good performing machines to shiny, qubit machines, we will see how we prepare everything, therefore we will have a comprehensive understanding of the code. Let's see how we prepare the grand combat between these two computing paradigms, to be able to compare their strengths and weaknesses.

4.1 Import Python Libraries

The following libraries are used:

1. **Standard Libraries of Python:** The `os`, `time`, `warnings` for file operations, `time` and `warning`, and `clear_output` is used to clear the output of Jupyter notebook.
2. **Data and Math:** The `numpy` and `pandas` are used for data and math operations, and they are essential for data manipulation and analysis.
3. **Graphing:** The `matplotlib.pyplot` is used for drawing the graphs, thus providing a visual representation of the data.
4. **Classical Machine Learning:** The `scikit-learn` library was used for preparing the data and `Imbalanced-learn` is used for fixing class imbalance, including `SMOTE`, training the models and testing, therefore `LabelEncoder`, `MinMaxScaler`, `LogisticRegression`, `GaussianNB`, `SVC`, `KNeighborsClassifier`, `roc_auc_score`, and `classification_report` are used.
5. **Quantum Machine Learning:** The `qiskit` library was used as the main quantum computing library, as it provides the necessary tools for quantum computing, including Quantum circuits, algorithms, and models, such as `RealAmplitudes`, `ZZFeatureMap`, `ZFeatureMap`, `Sampler`, `COBYLA`, `VQC`, `PegasosQSVC`, `SamplerQNN`, `EstimatorQNN`, `RandomUnderSampler`, and `Qiskit Runtime` is used for IBM Quantum service, including `Session`, `QiskitRuntimeService`, `FakeManilaV2`.

```

1      # Standard libraries for qml comparison
2      import os
3      import time
4      import warnings
5      from IPython.display import clear_output
6      import numpy as np
7      import pandas as pd
8      import matplotlib.pyplot as plt
9      from sklearn.preprocessing import LabelEncoder, MinMaxScaler
10     from sklearn.model_selection import StratifiedKFold
11     from sklearn.linear_model import LogisticRegression
12     from sklearn.naive_bayes import GaussianNB
13     from sklearn.svm import SVC
14     from sklearn.neighbors import KNeighborsClassifier
15     from sklearn.metrics import roc_auc_score, classification_report
16     from imblearn.over_sampling import SMOTE
17     from imblearn.under_sampling import RandomUnderSampler
18     from qiskit.circuit.library import RealAmplitudes, ZZFeatureMap,
        ZFeatureMap
19     from qiskit.primitives import Sampler
20     # Qiskit Algorithms
21     from qiskit_algorithms.optimizers import COBYLA
22     from qiskit_algorithms.utils import algorithm_globals
23     # Qiskit Machine Learning
24     from qiskit_machine_learning.algorithms.classifiers import
        NeuralNetworkClassifier, VQC
25     from qiskit_machine_learning.neural_networks import SamplerQNN,
        EstimatorQNN
26     from qiskit_machine_learning.circuit.library import QNNCircuit
27     from qiskit_machine_learning.kernels import FidelityQuantumKernel
28     from qiskit_machine_learning.algorithms import PegasosQSVC
29     from collections import Counter
30     # Qiskit Runtime
31     from qiskit_ibm_runtime import Session, QiskitRuntimeService
32     from qiskit_ibm_runtime.fake_provider import FakeManilaV2

```

4.2 Setting Up the Environment

The `setup_environment` function is a crucial step in ensuring that the script runs smoothly and consistently. It configures the environment with necessary settings to avoid interruptions and maintain reproducibility:

```

1      def setup_environment():
2          """Set up the environment with necessary configurations."""
3          warnings.filterwarnings("ignore") # Suppresses unnecessary warnings
           for a cleaner output
4          pd.set_option("display.max_rows", None) # Ensures all rows of a
           DataFrame are displayed
5          plt.style.use('ggplot') # Sets a visually appealing style for plots
6          algorithm_globals.random_seed = 12345 # Fixes the random seed for

```

- **Suppressing Warnings:** By ignoring warnings, the script avoids cluttering the output with non-critical messages, allowing the user to focus on the results.
- **DataFrame Display:** Setting `display.max_rows` to `None` ensures that all rows of a `DataFrame` are visible, which is particularly useful when debugging or exploring datasets.
- **Plot Styling:** The `ggplot` style provides a clean and professional look for visualizations, making it easier to interpret results.
- **Reproducibility:** Fixing the random seed ensures that any stochastic processes (e.g., random splits in cross-validation) produce consistent results across runs.

4.3 Configuring the Script

The script includes a set of constants that allow users to customize the experiment for their specific dataset and requirements. These constants act as the control panel for the entire implementation:

```

1      # **Important:** Modify these constants to configure the script for
      your specific dataset and experiment.

2
3      DATASET_PATH = "datasets/heart.csv" # Path to your CSV dataset file
4      TARGET_COLUMN = "HeartDisease" # Name of the column you want to
      predict
5      FEATURE_COUNTS = [2,3,4,5,6,7,8,9,10,11] # List of feature counts
      to evaluate
6      CROSS_VALIDATION_FOLDS = 2 # Number of cross-validation folds
7      HANDLE_IMBALANCE = 'smote' # Method to handle class imbalance:
      'smote', 'undersample', or None
8      IMBALANCE_THRESHOLD = 0.7 # Ratio of majority/minority class to
      trigger imbalance handling
9      USE_REAL_QUANTUM = True # Set to True to use a real quantum
      computer (requires Qiskit Runtime setup)
10     OUTPUT_FILE = "results.csv" # Output file to save the results
11     VERBOSE = True # Set to True for more detailed output during
      execution
12     MY_IBM_TOKEN = os.environ.get('MY_IBM_QUANTUM_TOKEN') # Fetches the
      IBM Quantum token from environment variables

```

Listing 4.1: Core libraries of the implementation showstringspaces

- **Dataset Path and Target Column:** These constants specify the location of the dataset and the target variable to be predicted. In this case, the dataset is a CSV file containing heart disease data, and the target column is "HeartDisease".
- **Feature Counts:** A list of feature counts to evaluate allows the script to test the performance of models with varying numbers of features, enabling feature selection experiments.
- **Cross-Validation Folds:** The number of folds for cross-validation determines how the dataset is split for training and testing. A value of 2 ensures a simple yet effective evaluation.

- **Class Imbalance Handling:** The `HANDLE_IMBALANCE` constant specifies the method to address class imbalance. Options include:
 - 'smote': Synthetic Minority Oversampling Technique to generate synthetic samples for the minority class.
 - 'undersample': Randomly removes samples from the majority class.
 - None: No imbalance handling.
- **Imbalance Threshold:** This value determines when class imbalance handling is triggered. If the ratio of majority to minority class exceeds 0.7, the specified method is applied.
- **Quantum Hardware Usage:** The `USE_REAL_QUANTUM` flag allows users to toggle between simulated and real quantum hardware. When set to True, the script connects to IBM Quantum services.
- **Output File:** Results are saved to a CSV file for easy access and further analysis.
- **Verbose Mode:** When enabled, the script provides detailed output during execution, which is helpful for debugging and understanding the process.
- **IBM Quantum Token:** The token is securely fetched from environment variables, ensuring safe and seamless access to IBM Quantum services.

4.4 Setting Up IBM Quantum Service

To use IBM Quantum hardware, the script sets up the IBM Quantum service account. This step is essential for running quantum models on real quantum devices:

```

1      # Set up the IBM Quantum service account
2      QiskitRuntimeService.save_account(channel="ibm_quantum",
      token=MY_IBM_TOKEN, overwrite=True)

```

Listing 4.2: Set up IBM Quantum service account showstringspaces

- **Saving the Account:** The `save_account` method stores the IBM Quantum account credentials locally, allowing the script to authenticate and access quantum devices.
- **Channel and Token:** The `channel` parameter specifies the type of service (in this case, "ibm_quantum"), and the `token` parameter provides the authentication key.
- **Overwrite Option:** Setting `overwrite=True` ensures that any existing account credentials are replaced with the new ones, avoiding conflicts.

This setup enables seamless integration with IBM Quantum services, allowing the script to execute quantum circuits on real hardware or simulators.

4.5 Data Preprocessing

The data preprocessing stage is a fundamental step in preparing the dataset for both classical and quantum machine learning models. The `load_and_preprocess_data` function was implemented to handle various preprocessing tasks, including loading the dataset, handling missing values, encoding categorical features, scaling numerical features, and detecting class imbalance. This section provides a detailed explanation of the preprocessing pipeline.

4.5.1 Loading the Dataset

The dataset is loaded from a specified file path using the pandas library. The function ensures that the dataset exists and that the target column is present. If the dataset file is not found or the target column is missing, appropriate error messages are raised to guide the user.

```
1 df = pd.read_csv(dataset_path)
2 if target_column not in df.columns:
3     raise ValueError(f"Target column '{target_column}' not found in dataset.")
```

Listing 4.3: Read the desired dataset showstringspaces

This step ensures that the dataset is correctly loaded and ready for further processing.

4.5.2 Handling Missing Values

The function employs a flexible approach to handle missing values, depending on the size of the dataset and the type of the column. For large datasets (more than 10,000 rows), rows with missing values are dropped to reduce computational overhead. For smaller datasets, missing values in categorical columns are filled with the most frequent value (mode), while missing values in numerical columns are filled with the median.

```
1 for col in df.columns:
2     if df[col].isnull().any():
3         if df.shape[0] > 10000:
4             df.dropna(inplace=True)
5         elif df[col].dtype == 'object' or df[col].dtype == 'string':
6             df[col] = df[col].fillna(df[col].mode()[0])
7         else:
8             df[col] = df[col].fillna(df[col].median())
```

Listing 4.4: Handling missing values showstringspaces

This approach ensures that missing values are handled appropriately without compromising the integrity of the dataset.

4.5.3 Encoding Categorical Features

Categorical columns are identified and encoded into numerical representations using the LabelEncoder from the scikit-learn library. Before encoding, the function ensures type consistency by converting categorical columns to strings.

```
1 for col in df.columns:
2     if df[col].dtype == 'object' or df[col].dtype == 'string':
3         df[col] = df[col].astype(str)
4         le = LabelEncoder()
5         df[col] = le.fit_transform(df[col])
```

Listing 4.5: Encoding features showstringspaces

This step transforms categorical data into a format suitable for machine learning models while preserving the information contained in the original features.

4.5.4 Detecting Class Imbalance

The function calculates the class distribution of the target variable and computes the imbalance ratio, defined as the proportion of the majority class to the total number of samples. If the imbalance ratio exceeds a specified threshold (default: 0.7), the function flags the dataset as imbalanced and prompts the user to specify a handling method, such as SMOTE or undersampling. This step ensures that class imbalance is detected early, allowing for appropriate corrective measures to be applied.

```
1 class_counts = Counter(y)
2 majority_class_count = max(class_counts.values())
3 imbalance_ratio = majority_class_count / y.shape[0]
4
5 if imbalance_ratio > imbalance_threshold:
6     print(f"Significant class imbalance detected (ratio:
7         {imbalance_ratio:.2f}).")
8     print("Please specify the 'HANDLE_IMBALANCE' constant ('smote' or
9         'undersample').")
```

Listing 4.6: Imbalancing detection in the dataset showstringspaces

4.5.5 Scaling Numerical Features

To normalize the feature values and ensure compatibility with quantum models, the function applies Min-Max scaling using the `MinMaxScaler` from the `scikit-learn` library. This scaling method transforms the feature values to a range between 0 and 1. This step is particularly important for quantum models, as quantum circuits are sensitive to the range of input values.

```
1 scaler = MinMaxScaler()
2 X = scaler.fit_transform(X)
```

Listing 4.7: Data scaling showstringspaces

4.6 Evaluating Classical Machine Learning Models

The evaluation of classical machine learning models was conducted using a systematic approach to ensure robust and reliable results. The process involved stratified k-fold cross-validation, which maintains the class distribution across folds, and the computation of key performance metrics such as ROC AUC scores, precision, recall, and F1-score. The following steps outline the evaluation process:

4.6.1 Workflow of Model Evaluation

1. **Initializing Results:** An empty dictionary was created to store the results for each model.
2. **Iterating Over Models:** Each model was trained and validated using stratified k-fold cross-validation to ensure balanced class distributions.
3. **Training and Validation:** Models were trained on the training set and validated on the validation set. The ROC AUC score was computed for each fold.
4. **Computing Average Scores:** The average ROC AUC score across all folds was calculated and stored.

5. **Classification Reports:** Detailed classification reports, including precision, recall, and F1-score, were generated for each fold.

4.6.2 Python Code for Model Evaluation

The following Python code snippets demonstrate the implementation of the evaluation process:

```
1  from sklearn.model_selection import StratifiedKFold
2  from sklearn.metrics import roc_auc_score, classification_report
3  import numpy as np
4
5  def evaluate_models(X, y, num_features, cross_validation_folds=2):
6      """Main function to evaluate all models."""
7      # Define classical models
8      models = {
9          "Logistic Regression": LogisticRegression(),
10         "Naive Bayes": GaussianNB(),
11         "SVM (Linear Kernel)": SVC(kernel="linear"),
12         "SVM (RBF Kernel)": SVC(kernel="rbf"),
13         "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=32),
14     }
15
16     # Setup cross-validation
17     kf = StratifiedKFold(n_splits=cross_validation_folds, shuffle=True,
18                          random_state=42)
19     results = {}
20
21     # Evaluate classical models
22     results.update(evaluate_classical_models(X, y, models, kf))
```

Listing 4.8: Python code for evaluating classical machine learning models

At first, the models from Scikit-learn are initialized with no fancy hyperparameter tuning. Then the cross validation is initialized and the start of the evaluation of the classical models starts.

```
1  from sklearn.model_selection import StratifiedKFold
2  from sklearn.metrics import roc_auc_score, classification_report
3  import numpy as np
4
5  def evaluate_classical_models(models, X, y):
6      results = {}
7      kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
8
9      for model_name, model in models.items():
10         fold_scores = []
11         for train_idx, val_idx in kf.split(X, y):
12             X_train, X_val = X[train_idx], X[val_idx]
13             y_train, y_val = y[train_idx], y[val_idx]
14
15             model.fit(X_train, y_train)
16             y_pred = model.predict(X_val)
17             score = roc_auc_score(y_val, y_pred)
18             fold_scores.append(score)
19
```

```

20     results[model_name] = np.mean(fold_scores)
21     print(f"{model_name} - Average ROC AUC Score:
        {results[model_name]:.4f}")
22     print(classification_report(y_val, y_pred))
23
24     return results

```

Listing 4.9: Python code for evaluating classical machine learning models

The **results** variable serves as our central repository, storing all the key metric scores for further comparisons later.

4.7 Quantum Machine Learning Implementation Analysis

In this section, we take a closer look at the code that brings our quantum machine learning experiments to life. Let's break down the main components, understand their roles, and discuss the rationale behind each choice.

4.7.1 Setting the Quantum Stage

Before any quantum magic can happen, we need to set up our quantum environment. This involves defining how classical data is encoded into quantum states, choosing a quantum circuit (ansatz) for learning, and selecting an optimizer to train our model. The following code snippet illustrates this setup:

```

1 feature_map = ZZFeatureMap(feature_dimension=X.shape[1], reps=1)
2 ansatz = RealAmplitudes(num_qubits=X.shape[1], reps=3)
3 optimizer = COBYLA(maxiter=100)
4 sampler = Sampler()

```

Listing 4.10: Quantum Setup

- **ZZFeatureMap:** This quantum feature map encodes classical data into quantum states using entangling gates. Setting `reps=1` means we perform a single round of encoding, which keeps the circuit depth manageable (and the quantum computer happy).
- **RealAmplitudes Ansatz:** This is our quantum circuit template, with three repetitions (`reps=3`) to give the model enough expressivity to learn complex patterns.
- **COBYLA Optimizer:** A classical optimizer that doesn't require gradient information, making it suitable for noisy quantum environments.
- **Sampler:** This is our measurement tool, allowing us to extract classical information from quantum states.

4.7.2 Choosing the Quantum Backend

To run our quantum algorithms, we need access to quantum hardware (or a good simulator). The code below shows how we select our backends:

```

1 service = QiskitRuntimeService(instance="ibm-q/open/main")
2 backend = service.least_busy(operational=True, simulator=False,
    min_num_qubits=10)
3 simulator_backend = FakeManilaV2()

```

Listing 4.11: Backend Selection

Here, we connect to IBM’s quantum cloud service and automatically select the least busy real quantum device with at least 10 qubits in order to run our QML models in a real quantum computer. For simulation, we use FakeManilaV2, which mimics the behavior of a real IBM quantum processor.

When we first started playing around with these quantum machine learning models, we used a local quantum simulator. It was great for small-scale experiments, but as we started needing more and more qubits, things got **really** slow. Imagine trying to run a super complex video game on a basic computer – that’s what it felt like!

To speed things up, we switched to using what’s called a “fake quantum computer backend.” Now, don’t let the name fool you – it’s not **actually** a quantum computer, but it’s a clever simulation that mimics the real one.

Think of it like this: Qiskit has a module called FakeProvider that’s like a box of toy quantum computers. These “fake backends” are designed to act just like IBM’s real quantum systems. They come with all the important details, like how the qubits are connected, what operations they can do, and even how noisy they are (including T_1 , T_2 , and error rates).

Using these fake backends let us test our algorithms and see how they might perform on real quantum hardware without having to wait in long queues or deal with the full complexity of actual quantum devices.

4.7.3 Data Splitting and Model Evaluation

We use the last split from a cross-validation procedure to separate our data into training and validation sets:

```

1 train_idx, val_idx = list(kf.split(X, y))[-1]
2 X_train, X_val = X[train_idx], X[val_idx]
3 y_train, y_val = y[train_idx], y[val_idx]

```

Listing 4.12: Data Splitting

With our data ready, we evaluate several quantum models by running them in both the simulator and the real quantum computer:

```

1 vqc_score = evaluate_vqc_model(X_train, y_train, X_val, y_val,
    feature_map, ansatz, optimizer, sampler, service, simulator_backend)
2 results["Quantum VQC"] = vqc_score
3
4 results["Real Quantum VQC"] = evaluate_real_quantum_vqc(
5     X_train, y_train, X_val, y_val, feature_map, ansatz, optimizer,
6     service, backend)
7
8 feature_map_peg = ZFeatureMap(feature_dimension=X.shape[1], reps=1)
9 results["PegasosQSVC"] = evaluate_pegasos_model(
10     X_train, y_train, X_val, y_val, feature_map_peg, service,
11     simulator_backend)

```

```

10 results["Real Quantum PegasosQSVC"] = evaluate_real_quantum_pegasos(
11     X_train, y_train, X_val, y_val, feature_map_peg, service, backend)
12
13 results["Estimator QNN"] = evaluate_estimator_qnn_classifiers(X_train,
14     y_train, X_val, y_val, service, simulator_backend)
15 results["Real Quantum Estimator QNN"] =
16     evaluate_real_estimator_qnn(X_train, y_train, X_val, y_val, service,
17     backend)
18 results["Sampler QNN"] = evaluate_sampler_qnn_classifiers(X_train,
19     y_train, X_val, y_val, service, simulator_backend)
20 results["Real Quantum Sampler QNN"] = evaluate_real_sampler_qnn(X_train,
21     y_train, X_val, y_val, service, backend)

```

Listing 4.13: Model Evaluation

4.7.4 QML models analysis

We evaluate and compare the following quantum models:

- **Variational Quantum Classifier (VQC):** The VQC is a hybrid quantum-classical algorithm designed for classification tasks. It operates by first encoding classical data into quantum states using the **ZZFeatureMap**, which captures correlations between features through entanglement. The heart of the VQC is a parameterized quantum circuit, known as a variational ansatz, whose parameters are optimized using classical optimization techniques. This flexible structure allows the VQC to adaptively learn complex decision boundaries, making it a versatile tool for a variety of classification problems.
- **PegasosQSVC:** is a quantum-enhanced adaptation of the classical Pegasos support vector classifier. By utilizing the **ZFeatureMap**, it projects data into a high-dimensional quantum feature space, where the separation between classes can become more pronounced. The quantum version leverages the potential of quantum computing to efficiently handle these large feature spaces, which may be intractable for classical algorithms. This approach aims to exploit quantum parallelism to improve the efficiency and possibly the accuracy of support vector classification.
- **Quantum Neural Networks (QNN):** Quantum Neural Networks represent a quantum analogue of classical neural networks, with the potential to capture intricate patterns in data. In this study, both estimator-based and sampler-based QNNs are explored. Estimator-based QNNs focus on evaluating expectation values of quantum observables, providing a direct measure of the network's output. Sampler-based QNNs, on the other hand, rely on sampling from the probability distribution generated by the quantum circuit. This dual approach allows for a comprehensive assessment of QNNs' capabilities in learning and generalization, as well as their sensitivity to quantum noise

Each of these models is evaluated on both a quantum simulator and actual quantum hardware. This dual evaluation strategy enables a thorough investigation of how quantum noise and hardware imperfections affect model performance, facilitates a comparison between idealized and real-world results, and helps identify which models demonstrate greater robustness in practical quantum computing environments.

Variational Quantum Classifier (VQC)

The Variational Quantum Classifier represents one of the most intuitive approaches to quantum machine learning, following a structure similar to classical neural networks but leveraging quantum mechanics' unique properties. Let's dive into our implementation:

```
1 def evaluate_vqc_model(X_train, y_train, X_val, y_val, feature_map, ansatz,
2                        optimizer, sampler, service, backend):
3     """Evaluate the Variational Quantum Classifier (VQC) model."""
4     vqc = VQC(sampler=sampler, feature_map=feature_map,
5               ansatz=ansatz, optimizer=optimizer)
6
7     with Session(service=service, backend=backend):
8         print("Training on simulated quantum hardware...")
9         start_time = time.time()
10        vqc.fit(X_train, y_train)
11        elapsed_time = time.time() - start_time
12        y_pred = vqc.predict(X_val)
13        print(f"Training completed in {elapsed_time:.2f} seconds.")
14
15        test_score = vqc.score(X_val, y_val)
16    print(classification_report(y_val, y_pred))
17
18    return np.mean(test_score)
```

Listing 4.14: VQC Implementation

Core Components

The VQC implementation consists of several crucial components working in harmony:

- **Feature Map (ZZFeatureMap):**
 - Encodes classical data into quantum states
 - Uses entangling ZZ gates to capture feature interactions
 - Single repetition (reps=1) balances expressivity with circuit depth

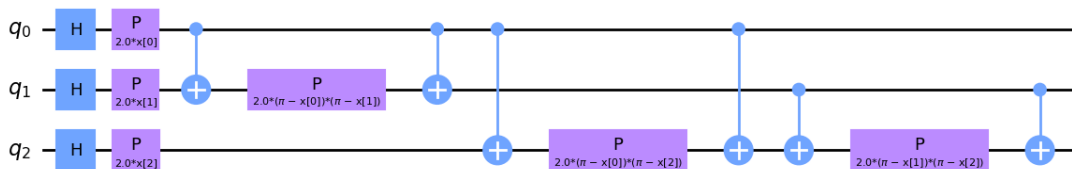


Figure 4.1: ZZFeatureMap for 3 features and single repetition

How does the ZFeatureMap work?

Abstract explanation of quantum feature maps

Quantum feature maps are the quantum analog of classical feature transformations, designed to embed classical data into the high-dimensional Hilbert space of a quantum computer. The

choice of feature map is crucial, as it determines how data is represented and what kinds of patterns a quantum machine learning model can capture. The **ZFeatureMap** is a quantum feature map used to encode classical data into quantum states, particularly for quantum machine learning algorithms such as QSVM and VQC that are implemented in this thesis. It is called “ZFeatureMap” because it uses rotations around the Pauli-Z axis to encode each feature of the data [24, 25, 26].

Suppose we have a classical data vector $\mathbf{x} = (x_1, x_2, \dots, x_n)$ and n qubits. The ZFeatureMap encodes each feature x_j into the j -th qubit by applying a rotation around the Z axis:

$$U_{\text{ZFeatureMap}}(\mathbf{x}) = \bigotimes_{j=1}^n R_z(x_j)$$

where $R_z(x_j)$ is the rotation gate defined as:

$$R_z(x_j) = \exp\left(-i\frac{x_j}{2}Z\right)$$

and Z is the Pauli-Z operator.

Table 4.1: Comparison of Quantum Feature Maps

Feature Map	Entanglement	Circuit Depth	Expressivity	Use Case
ZFeatureMap	No	Low	Low	Simple, fast encoding
ZZFeatureMap	Yes	Medium	Medium	Captures correlations
PauliFeatureMap	Yes	High	High	Most expressive

- **ZFeatureMap:** This map encodes each classical feature as a rotation around the Z axis of a qubit, with no entanglement between qubits. It is simple, shallow, and suitable for NISQ devices, but its expressivity is limited to linear or very basic nonlinear relationships.
- **ZZFeatureMap:** This map introduces entanglement by applying controlled-Z (ZZ) interactions between pairs of qubits, allowing the encoding of pairwise feature correlations. It is more expressive than the ZFeatureMap and can capture more complex data structures, but at the cost of increased circuit depth.
- **PauliFeatureMap:** This is the most general and expressive of the three, using a combination of single-qubit rotations and multi-qubit Pauli interactions. It can represent highly complex and nonlinear relationships in the data, but requires deeper circuits and is more sensitive to quantum noise.

Summary

The ZFeatureMap is a simple way to encode classical data into quantum states by rotating each qubit around the Z axis according to its feature value. It is fast and NISQ-friendly, but for more complex data relationships, feature maps that introduce entanglement may be preferred [8].

- **Ansatz (RealAmplitudes):**

- Parameterized quantum circuit that learns the classification boundary
- Repetitions provide the desired expressivity
- Uses real-valued rotation gates for optimization

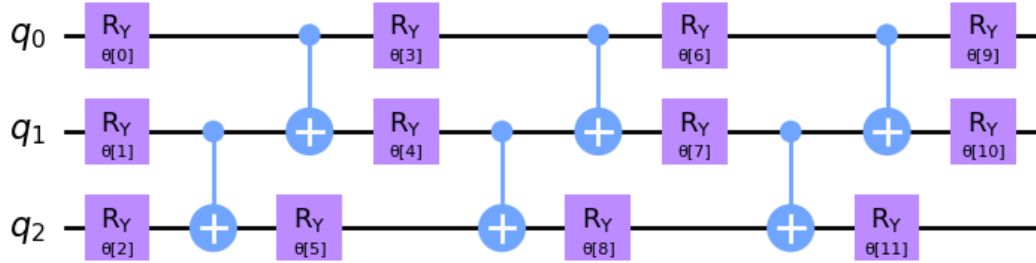


Figure 4.2: RealAmplitudes for 3 features and 3 repetitions

What is an Ansatz in Quantum Machine Learning?

In quantum machine learning, an ansatz refers to a parameterized quantum circuit designed to approximate a target quantum state or to solve a specific computational problem. Essentially, it acts as a flexible template whose parameters can be tuned during the training process to optimize the model's performance. The choice of ansatz is crucial, as it determines the expressiveness and trainability of the quantum model [27]. A well-chosen ansatz can efficiently represent complex functions, while a poorly chosen one may limit the model's ability to learn or lead to optimization difficulties such as barren plateaus (regions of flat gradients) [28].

For this thesis, the RealAmplitudes class from Qiskit is used to construct the variational ansatz. The RealAmplitudes ansatz is widely adopted due to its simplicity, hardware efficiency, and proven effectiveness in various quantum machine learning tasks. It consists of layers of parameterized single-qubit rotations and entangling gates, allowing it to capture a broad range of quantum states while remaining relatively easy to implement on current quantum hardware [8].

- **Optimizer (COBYLA):**

- Gradient-free optimization algorithm
- Well-suited for noisy quantum environments
- Maximum iterations set to 100 for practical convergence

What is COBYLA algorithm?

COBYLA (Constrained Optimization BY Linear Approximation) is a gradient-free optimization algorithm that is particularly well-suited for problems where the objective function is noisy, non-differentiable, or expensive to evaluate—conditions that are often encountered in quantum machine learning [29]. Unlike gradient-based optimizers, COBYLA does not require the calculation of derivatives. Instead, it builds linear approximations of the objective function and iteratively searches for the minimum by solving a series of trust-region subproblems.

In the context of variational quantum algorithms, such as the Variational Quantum Classifier (VQC), the cost function landscape can be rugged and affected by quantum noise, making gradient-free methods like COBYLA especially attractive. COBYLA is robust, easy to use, and often performs well for small- to medium-sized parameter spaces, which are common in near-term quantum devices.

In this thesis, the COBYLA optimizer is imported from *qiskit_algorithms.optimizers* and is used to tune the parameters of the variational ansatz, seeking to minimize the classification error.

- **Sampler:**

- Performs quantum measurements
- Converts quantum states to classical predictions
- Handles shot-based statistics

What is a Sampler?

Quantum measurements are performed using a sampler, which executes the quantum circuit multiple times and collects the resulting measurement outcomes. This process is essential for extracting meaningful information from quantum computations, as the probabilistic nature of quantum mechanics means that each run of the circuit can yield different results. By aggregating these outcomes, the sampler enables the estimation of probabilities and expectation values required for model evaluation [11]. This approach ensures consistent and reliable extraction of model predictions, facilitating the calculation of performance metrics and the comparison of different quantum models [30].

In this thesis, the Qiskit Sampler primitive is used to obtain measurement results from the quantum circuits, enabling the evaluation of model predictions and the calculation of performance metrics [8].

VQC Implementation on Real Quantum Hardware

After evaluating the VQC model in a simulated environment, we extend our analysis to real quantum hardware, providing crucial insights into the model's practical performance. Here's our implementation:

```
1 def evaluate_real_quantum_vqc(X_train, y_train, X_val, y_val, feature_map,
2                               ansatz, optimizer, service, backend):
3     """Evaluate VQC model on real quantum hardware."""
4     vqc = VQC(sampler=Sampler(), feature_map=feature_map,
5               ansatz=ansatz, optimizer=optimizer)
6
7     with Session(service=service, backend=backend):
8         print("Training on real quantum hardware...")
9         start_time = time.time()
10        vqc.fit(X_train, y_train)
11        elapsed_time = time.time() - start_time
12        y_pred = vqc.predict(X_val)
13        print(f"Training completed in {elapsed_time:.2f} seconds.")
14
15        test_score = vqc.score(X_val, y_val)
16        print(classification_report(y_val, y_pred))
```

```

17
18     return test_score

```

Listing 4.15: VQC Implementation on Real Quantum Hardware

As shown in Listing 4.15, the core logic of the *evaluate_real_quantum_vqc* function is identical to its simulated counterpart. The primary difference lies in the backend parameter, which now specifies a real quantum device accessible through the IBM Quantum cloud service. This allows us to assess the impact of real-world quantum effects on the VQC’s performance.

Pegasos QSVM (Quantum Support Vector Machine)

The original Pegasos algorithm addresses the following primal SVM optimization problem:

$$\min_w \left[\frac{\lambda}{2} |w|^2 + \frac{1}{m} \sum_{i=1}^m \ell(w; (x_i, y_i)) \right],$$

where $\ell(w; (x, y)) = \max\{0, 1 - y\langle w, x \rangle\}$ is the hinge loss, λ is the regularization parameter, and (x_i, y_i) are the training samples. Pegasos operates by iteratively updating the weight vector w using stochastic sub-gradient descent, selecting a random training example (or mini-batch) at each step. The algorithm is notable for its efficiency, as its runtime depends primarily on the regularization parameter and the desired accuracy, rather than the size of the training set.

Quantum Kernel Methods

In the quantum setting, the Pegasos algorithm is combined with a quantum kernel, which is computed using a quantum computer. The quantum kernel is defined as:

$$K(x, x') = |\langle \psi(x) | \psi(x') \rangle|^2,$$

where $|\psi(x)\rangle$ is the quantum state encoding the classical data point x via a parameterized quantum circuit (feature map). This kernel captures the similarity between data points in a high-dimensional Hilbert space, potentially enabling the classifier to separate data that is not linearly separable in the original feature space.

Why Quantum Kernels?

Classical kernel methods are powerful, but they are ultimately limited by the types of patterns they can recognize—patterns that are, in a sense, “classically computable.” Quantum kernel methods, on the other hand, leverage the unique properties of quantum mechanics—such as superposition and entanglement—to map data into exponentially larger feature spaces. This opens up the tantalizing possibility of capturing complex, non-classical correlations that are inaccessible to classical algorithms [31].

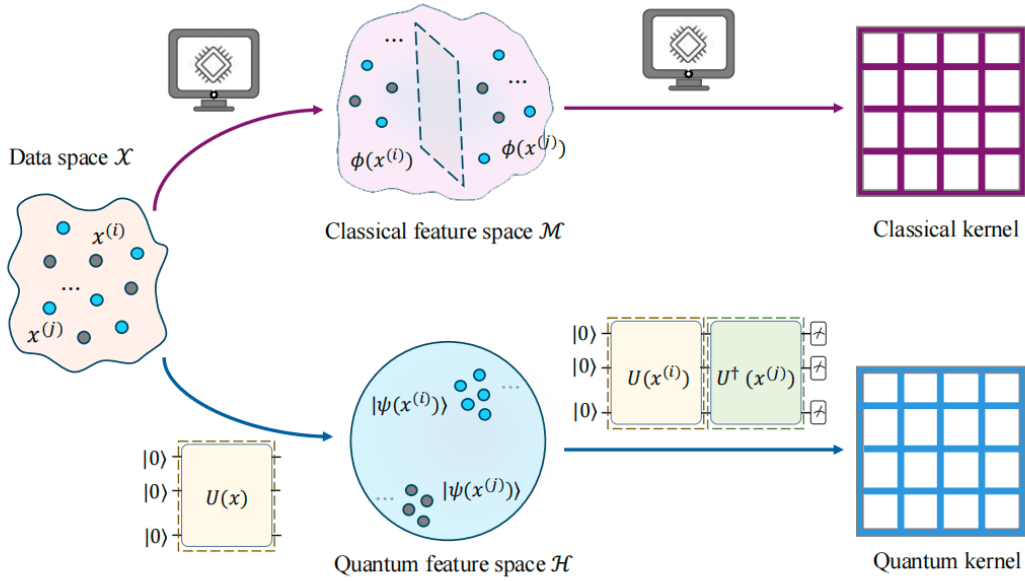


Figure 4.3: The paradigm of classical and quantum kernels [12]

Both of the classical and quantum kernels embed the data points from the data space $\mathcal{X}$ into high-dimensional space, and then compute the kernel as the inner product of the feature maps. The quantum kernel leverages quantum circuits to achieve this goal, as indicated by the blue color.

PegasosQSVC Implementation in Qiskit

The Qiskit Machine Learning library provides an implementation of PegasosQSVC, which integrates the Pegasos optimization routine with a quantum kernel. The typical workflow is as follows:

```

1 def evaluate_pegasos_model(X_train, y_train, X_val, y_val, feature_map,
2   service, backend):
3     """Evaluate the Pegasos Quantum Support Vector Classifier (QSVC) model."""
4     feature_map = ZFeatureMap(feature_dimension=X.shape[1], reps=1)
5
6     pegasos_qsvc =
7     PegasosQSVC(quantum_kernel=QuantumKernel(feature_map=feature_map),
8                 optimizer=COBYLA(maxiter=100),
9                 reg_param=0.01)
10
11     with Session(service=service, backend=backend):
12         print("Training PegasosQSVC on simulated quantum hardware...")
13         start_time = time.time()
14         pegasos_qsvc.fit(X_train, y_train)
15         elapsed_time = time.time() - start_time
16         y_pred = pegasos_qsvc.predict(X_val)
17         print(f"Training completed in {elapsed_time:.2f} seconds.")
18
19         test_score = pegasos_qsvc.score(X_val, y_val)
20         print(f"Test Score: {test_score:.4f}")
21         print("-----")
22         print(classification_report(y_val, y_pred))

```

Listing 4.16: PegasosQSVC Implementation

Key Components and Workflow

- **Quantum Feature Map:** The `ZFeatureMap` or other parameterized circuits are used to encode classical data into quantum states. The choice of feature map determines the structure of the quantum kernel.
- **Quantum Kernel:** The `QuantumKernel` class in Qiskit computes the kernel matrix by executing quantum circuits on a simulator or real quantum hardware. This matrix is then used by the Pegasos algorithm to find the optimal separating hyperplane.
- **Pegasos Optimizer:** The core optimization is performed using stochastic sub-gradient descent, as in the classical Pegasos algorithm, but with the kernel matrix computed quantumly.
- **Regularization:** The regularization parameter λ (or `reg_param`) controls the trade-off between maximizing the margin and minimizing classification error.

Advantages and Considerations

- **Scalability:** PegasosQSVC is particularly well-suited for large datasets, as each iteration operates on a single (or a small batch of) training example(s), and the quantum kernel can be computed in parallel.
- **Quantum Advantage:** By leveraging quantum feature spaces, PegasosQSVC may be able to classify data that is intractable for classical SVMs, provided the quantum kernel captures relevant structure in the data.
- **Practical Limitations:** The efficiency of PegasosQSVC in practice depends on the availability of quantum hardware, the noise characteristics of the device, and the overhead of kernel matrix computation.
- **Empirical Performance:** As shown in [32] and in Qiskit's tutorials [33], Pegasos-based methods can achieve competitive accuracy with significantly reduced training times, especially for sparse or high-dimensional data.

QSVM Implementation on Real Quantum Hardware

```

1 def evaluate_real_quantum_pegasos(X_train, y_train, X_val, y_val, feature_map,
2     service, backend, C=1000, tau=100):
3     """Evaluate PegasosQSVC model on real quantum hardware."""
4     qkernel = FidelityQuantumKernel(feature_map=feature_map)
5     pegasos_qsvc = PegasosQSVC(quantum_kernel=qkernel, C=C, num_steps=tau)
6
7     with Session(service=service, backend=backend):
8         print("Training on real quantum hardware...")
9         start_time = time.time()
10        pegasos_qsvc.fit(X_train, y_train)
11        elapsed_time = time.time() - start_time
12        y_pred = pegasos_qsvc.predict(X_val)

```

```

13     print(f"Training completed in {elapsed_time:.2f} seconds.")
14
15     pegasos_score = pegasos_qsvc.score(X_val, y_val)
16     print(f"Test Score: {pegasos_score:.4f}")
17     print("-----")
18     print(classification_report(y_val, y_pred))
19     return pegasos_score

```

Listing 4.17: PegasosQSVC Implementation

As in the VQC implementation in the Real Quantum Hardware, the code is exactly the same with the only difference to be the backend that point to the IBM's cloud that runs our code to the real quantum computer.

Summary

The PegasosQSVC algorithm represents a promising approach to quantum machine learning, combining the theoretical efficiency of stochastic sub-gradient descent with the expressive power of quantum kernels. Its implementation in Qiskit enables experimentation on both simulators and real quantum hardware, providing a valuable tool for exploring the potential of quantum-enhanced classification.

4.7.5 Quantum Neural Networks (QNNs)

Quantum Neural Networks represent a fascinating intersection of quantum computing and neural network architectures. In our implementation, we explore two distinct types of QNNs: Estimator-based QNNs and Sampler-based QNNs, each offering unique approaches to quantum machine learning.

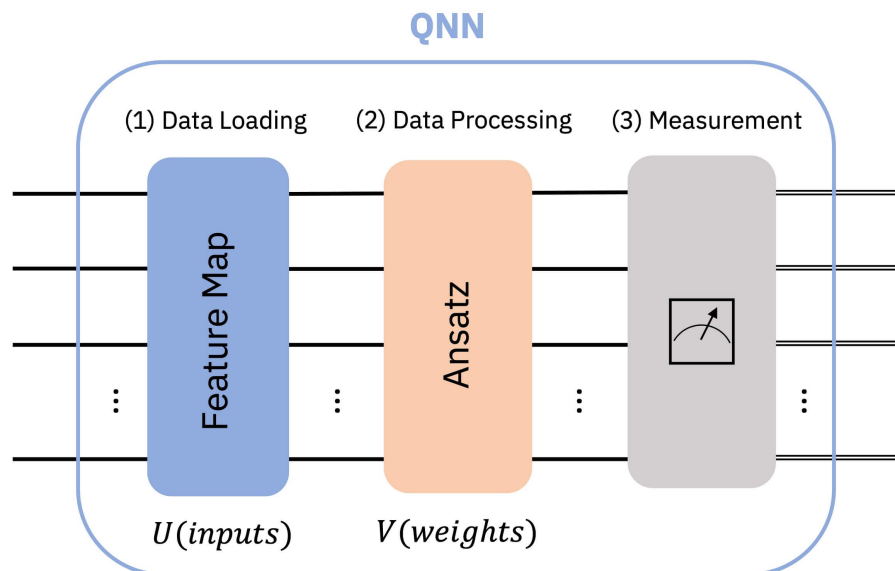


Figure 4.4: Generic Quantum Neural Network (QNN) structure [34]

Architecture Overview

Our QNN implementations utilize Qiskit's machine learning framework and consist of three main components which look quite similar to the VQC ones:

- **Quantum Circuit:** The core quantum processing unit
- **Ansatz:** RealAmplitudes for sampler and EstimatorQNN with random weights for estimator.
- **Measurement Interface:** Either Estimator or Sampler for quantum state readout

Estimator-based QNN

The Estimator QNN uses expectation values of quantum measurements for its predictions. Here's the implementation:

```
1 def create_estimator_qnn(X):
2     """Create and configure EstimatorQNN."""
3     qc = QNNCircuit(num_qubits=X.shape[1])
4     estimator_qnn = EstimatorQNN(circuit=qc)
5     # Test forward pass
6     estimator_qnn.forward(X[0, :],
7         algorithm_globals.random.random(estimator_qnn.num_weights))
8     return estimator_qnn
9
10 def evaluate_estimator_qnn_classifiers(X_train, y_train, X_val, y_val,
11     service, backend, max_iterations=30):
12     """Evaluate EstimatorQNN classifier."""
13     estimator_qnn = create_estimator_qnn(X_train)
14
15     estimator_classifier = NeuralNetworkClassifier(
16         estimator_qnn,
17         optimizer=COBYLA(maxiter=max_iterations),
18     )
19
20     with Session(service=service, backend=backend):
21         print("Training Estimator QNN on simulated quantum hardware...")
22         start_time = time.time()
23         estimator_classifier.fit(X_train, y_train)
24         elapsed_time = time.time() - start_time
25         y_pred = estimator_classifier.predict(X_val)
26
27         e_qnn_score = estimator_classifier.score(X_val, y_val)
28
29     return e_qnn_score
```

Listing 4.18: Estimator QNN Implementation

Key Features of Estimator QNN:

- **Quantum Circuit:** Uses a parameterized quantum circuit with number of qubits matching the input dimension
- **Measurement:** Directly estimates expectation values of quantum observables
- **Output:** Produces continuous-valued outputs suitable for classification

Sampler-based QNN

The Sampler QNN takes a different approach, using quantum measurement outcomes and a classical post-processing step:

```
1 def create_sampler_qnn(X):
2     """Create and configure SamplerQNN with parity mapping."""
3     qc = QNNCircuit(ansatz=RealAmplitudes(X.shape[1], reps=1))
4
5     def parity(x):
6         """Maps bitstrings to 0 or 1 based on number of 1s."""
7         return "{:b}".format(x).count("1") % 2
8
9     output_shape = 2 # corresponds to number of classes
10    sampler_qnn = SamplerQNN(
11        circuit=qc,
12        interpret=parity,
13        output_shape=output_shape,
14    )
15    return sampler_qnn
```

Listing 4.19: Sampler QNN Implementation

Key Features of Sampler QNN:

- **Quantum Circuit:** Employs `RealAmplitudes` ansatz for state preparation
- **Parity Mapping:** Uses a classical post-processing function to interpret measurement outcomes
- **Discrete Output:** Natural for classification tasks through bitstring measurements

Training Process

Both QNN variants share a similar training workflow:

The training procedure for both Quantum Neural Network (QNN) variants follows a structured and systematic workflow designed to ensure effective learning and robust evaluation.

The process begins with Initialization, where the quantum circuit is configured with the appropriate number of qubits to match the complexity of the problem at hand. The classical optimizer, specifically COBYLA, is set up with a predefined maximum number of iterations to balance convergence and computational efficiency. Network parameters are initialized randomly, providing a diverse starting point for the optimization process.

The core of the training is the Training Loop. In each iteration, a forward pass is performed by executing the quantum circuit with the current set of parameters. The model's predictions are then compared to the true labels to compute the loss, which quantifies the discrepancy between predicted and actual outcomes. The COBYLA optimizer is employed to update the network parameters, iteratively seeking to minimize the loss. Throughout this process, key performance metrics are tracked to monitor the model's progress and guide further optimization.

Finally, the Evaluation phase assesses the trained model's generalization ability. The QNN is used to predict labels on a separate validation set, and its performance is quantified using accuracy and other relevant classification metrics. A comprehensive classification report is generated, offering detailed insights into the strengths and potential areas for improvement of the model.

This structured approach ensures that both QNN variants are trained and evaluated in a consistent and rigorous manner, facilitating meaningful comparisons and reliable conclusions.

Implementation Considerations

Several key considerations guided the implementation of the Quantum Neural Network (QNN) models, ensuring both methodological rigor and practical feasibility.

Circuit Design was tailored to the specific requirements of each QNN variant. The estimator-based QNN utilizes a custom QNNCircuit, allowing for flexible circuit architecture and precise control over the quantum operations. In contrast, the sampler-based QNN employs the RealAmplitudes ansatz with a single repetition, striking a balance between expressivity and hardware efficiency.

The Measurement Strategy also differs between the two approaches. For the estimator QNN, model outputs are obtained directly as expectation values of quantum observables, providing a straightforward and noise-resilient measure of the circuit's predictions. The sampler QNN, on the other hand, relies on repeated bitstring measurements, with the final prediction determined by interpreting the parity of the measured outcomes. This approach leverages the probabilistic nature of quantum measurements to extract meaningful information from the circuit.

In terms of Optimization, both QNN variants employ the COBYLA optimizer, which is well-suited for the noisy and non-differentiable landscapes typical of quantum machine learning. The maximum number of optimization iterations is set to 30 by default, providing a practical trade-off between convergence and computational cost. Notably, the optimization process does not require explicit gradient computation, further simplifying the training procedure and enhancing compatibility with current quantum hardware.

These implementation choices collectively support the effective training and evaluation of the QNN models, while accommodating the unique challenges and opportunities presented by quantum computing.

Advantages and Limitations

The Quantum Neural Network (QNN) implementation offers several compelling advantages, balanced by certain limitations that reflect the current state of quantum computing technology.

On the positive side, the QNNs harness Quantum Expressivity, leveraging superposition and entanglement to potentially capture complex patterns in data that are difficult for classical models to represent. The Flexible Architecture supports different measurement strategies, allowing for customization based on the specific problem and hardware constraints. Furthermore, the QNNs are Hardware Compatible, capable of running on both quantum simulators and real quantum devices, facilitating experimentation and validation in diverse computing environments.

However, the QNN implementation also faces several limitations. The Circuit Depth is constrained by the coherence times of current quantum hardware, limiting the complexity of the quantum computations that can be performed. Training Time is increased by the overhead of quantum circuit execution, which can be significantly slower than classical computations. Finally, the classical optimizer used for parameter tuning may encounter challenges such as getting stuck in local optima, potentially hindering the model's ability to reach its full potential.

4.7.6 Technical Challenges and Considerations

Transitioning from quantum simulation to execution on real quantum hardware introduces a range of practical challenges and considerations that must be carefully addressed.

Real Hardware Considerations

A primary concern is the ever-present **Quantum Noise**. Unlike idealized simulators, real quantum devices are inherently susceptible to decoherence and gate errors, which can significantly impact

computational accuracy. As a result, the observed performance on hardware may deviate from simulation results, providing a more realistic reflection of the current limitations and capabilities of quantum technology.

Hardware Constraints also play a critical role in shaping the implementation. The **limited qubit count** restricts the size of problems we can tackle, requiring careful selection of devices with sufficient qubits. The limited coherence times of physical qubits impose restrictions on the allowable circuit depth, necessitating careful design to balance expressivity and feasibility. Additionally, the connectivity between qubits is often constrained by the hardware architecture, which can affect the mapping and execution of quantum circuits. Practical considerations such as **queue times** for accessing shared quantum devices (requiring us to select the least busy backend to minimize waiting) further influence the overall workflow.

Finally, **Execution Time** on real hardware encompasses several additional factors compared to simulation. The total training time includes not only the execution of quantum circuits but also the compilation process required to adapt circuits to the specific hardware. For cloud-based quantum computers, network latency can introduce further delays. Moreover, multiple measurement shots are typically required to obtain statistically reliable results, contributing to the overall execution time. Furthermore, **classical optimizers like COBYLA may struggle with noisy objective functions**, requiring careful tuning to achieve optimal performance.

By accounting for these real-world factors, the study provides a comprehensive evaluation of quantum machine learning models under both idealized and practical conditions.

Implementation Differences

Key differences from the simulated version include:

When transitioning from simulation to real quantum hardware, several key implementation differences arise that impact both workflow and results.

The first major difference concerns **Backend Selection**. While simulations typically utilize virtual backends such as FakeManilaV2, the real hardware implementation leverages actual IBM quantum devices. The system is designed to automatically select the least busy backend that meets the required qubit count, dynamically adapting to real-time hardware availability and queue conditions.

Another important distinction is the approach to **Error Mitigation**. On real hardware, implicit error correction is achieved through statistical analysis of measurement outcomes, with each circuit executed multiple times (shots) to average out the effects of noise. This strategy enhances robustness against hardware-specific noise patterns, helping to ensure more reliable results despite the inherent imperfections of current quantum devices.

Finally, the **Performance Metrics** collected on real hardware reflect practical, real-world conditions. These include accuracy measurements obtained from actual device runs, total training time that accounts for hardware overhead such as queueing and compilation, and an overall assessment of the practical feasibility of deploying quantum machine learning models on today's quantum computers.

By addressing these implementation differences, the study provides a realistic perspective on the challenges and opportunities associated with running quantum algorithms on real-world quantum hardware.

Practical Implications

Implementing quantum machine learning models on real quantum hardware yields a range of valuable practical insights that extend beyond what can be observed in simulation alone.

A central aspect is the **Quantum Advantage Assessment**. By directly comparing the performance of models on simulated and real quantum devices, it becomes possible to gauge the extent to which current hardware can deliver on the promise of quantum advantage. This comparison not only highlights the limitations imposed by today’s quantum technology but also helps to identify specific scenarios and use cases where quantum approaches may already offer practical benefits.

The study also sheds light on the **Resource Requirements** associated with real quantum computing. It provides concrete measurements of the actual quantum computing time needed for training and evaluation, as well as insights into hardware-specific optimization strategies that can enhance performance. Additionally, the results inform an assessment of scalability, offering a realistic perspective on the feasibility of applying quantum machine learning to larger datasets as quantum hardware continues to evolve.

Finally, the real hardware implementation points toward important **Future Directions**. The findings highlight areas where improvements in quantum hardware could have the greatest impact, as well as opportunities for further algorithmic optimization. Moreover, the experience gained suggests promising avenues for developing and integrating advanced error mitigation strategies, which will be essential for unlocking the full potential of quantum machine learning in practical applications.

4.7.7 Code and Reproducibility

Please refer to the repository’s README for environment setup, installation, and step-by-step instructions to run the experiments. The repository also includes a requirements file with all dependencies, scripts/notebooks to reproduce the reported results and figures, and notes on dataset preparation and quantum backend settings (e.g., shots and transpilation). Following the documented commands will allow you to replicate the classical baselines and quantum models exactly as presented in this thesis.

 https://github.com/asterios-pantousas/Qiskit_QML_Implementations

4.7.8 Summary

This implementation provides a comprehensive framework for benchmarking quantum machine learning models on both simulated and real quantum hardware. By systematically comparing different models and feature maps, we gain valuable insights into the current capabilities and limitations of quantum machine learning in practice. These insights, particularly regarding the impact of noise and hardware constraints, will be crucial in interpreting the results presented in the next chapter.

Chapter 5

Results and Discussion

5.1 Introduction

In this chapter, we present and analyze the performance of classical and quantum machine learning models across two benchmark datasets. The evaluation is based on accuracy, precision, recall, and F1 score, providing a comprehensive comparison between classical and quantum approaches, both in simulation and on real quantum hardware.

5.2 Experimental Setup

A brief recap of the experimental setup will be conducted to proceed with the results of the implementation.

Name	Number of Features	Target Name	Size	Has Imbalance
Heart Failure Prediction	12	HeartDisease	918	No
Stroke Prediction	11	stroke	5110	Yes

Table 5.1: Dataset brief information

5.2.1 Models

- **Classical Models:**

1. Logistic Regression
2. Naive Bayes
3. Support Vector Machines (SVM) - Linear Kernel
4. Support Vector Machines (SVM) - RBF Kernel
5. K-Nearest Neighbors (KNN)

- **Quantum Models:**

1. Variational Quantum Classifier (VQC)
2. QSVM Pegasos
3. Sampler/Estimator QNN

5.2.2 Evaluation Metrics

The following metrics are used for evaluation:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

5.2.3 Implementation Details

All experiments were conducted using a combination of classical and quantum computing resources to ensure a fair and comprehensive comparison between models.

For the classical machine learning models, training and evaluation were performed on a standard workstation equipped with an Apple M2 processor and 16 GB of RAM. All quantum algorithms were implemented using the `qiskit` library (version 1.2.4) implemented in Python (version 3.12.0).

For the quantum machine learning models, both simulated and real quantum hardware were utilized. Simulations were carried out using Qiskit’s `qiskit_ibm_runtime.fake_provider`, allowing for rapid prototyping and real like noise benchmarking. For real hardware experiments, IBM Quantum’s cloud-based devices were used, selected based on qubit availability and queue times at the time of execution.

Each quantum circuit was executed with 1024 shots (default value) to ensure statistically reliable measurement outcomes. The variational circuits used the `RealAmplitudes` ansatz with a single repetition, balancing expressivity and hardware constraints. The COBYLA optimizer, with a maximum of 30 iterations, was chosen for parameter optimization due to its robustness in noisy, gradient-free settings.

All experiments were managed using Qiskit’s `Session` and `Sampler` primitives, which facilitated seamless switching between simulators and real devices, as well as efficient resource management and job scheduling. Error mitigation was performed implicitly by averaging results over multiple shots and, where possible, by selecting the least busy backend to minimize the impact of hardware noise and queue delays.

This setup ensured that both classical and quantum models were evaluated under consistent and reproducible conditions, providing a solid foundation for the comparative analysis presented in this chapter.

5.3 Results

5.3.1 Heart Failure Prediction

Table 5.2: Comparison of Classical and Quantum Models (2 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time
Logistic Regression	<u>0.81</u>	<u>0.83</u>	0.80	<u>0.81</u>	Insignificant
Naive Bayes	<u>0.81</u>	<u>0.83</u>	0.80	<u>0.81</u>	Insignificant
Support Vector Machines (SVM) - Linear Kernel	<u>0.81</u>	0.81	<u>0.81</u>	<u>0.81</u>	Insignificant
Support Vector Machines (SVM) - RBF Kernel	<u>0.81</u>	0.81	0.80	0.80	Insignificant
K-Nearest Neighbors (KNN)	<u>0.81</u>	0.81	<u>0.81</u>	<u>0.81</u>	Insignificant
Variational Quantum Classifier (VQC)	<u>0.81</u>	<u>0.83</u>	0.80	<u>0.81</u>	Insignificant
QSVM Pegasos	0.80	0.80	0.80	0.80	Insignificant
Estimator QNN	0.51	0.19	0.31	0.23	Insignificant
Sampler QNN	<u>0.81</u>	0.81	<u>0.81</u>	<u>0.81</u>	Insignificant
Variational Quantum Classifier (VQC, real)	<u>0.81</u>	<u>0.83</u>	0.80	<u>0.81</u>	Insignificant
QSVM Pegasos (real)	<u>0.81</u>	<u>0.83</u>	0.80	<u>0.81</u>	Insignificant
Estimator QNN (real)	0.51	0.19	0.31	0.23	Insignificant
Sampler QNN (real)	<u>0.81</u>	0.81	<u>0.81</u>	<u>0.81</u>	Insignificant

Table 5.3: Comparison of Classical and Quantum Models (3 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time
Logistic Regression	0.81	<u>0.82</u>	0.80	0.80	Insignificant
Naive Bayes	0.77	0.77	0.77	0.77	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.81	0.81	<u>0.81</u>	<u>0.81</u>	Insignificant
Support Vector Machines (SVM) - RBF Kernel	<u>0.83</u>	<u>0.83</u>	0.82	0.82	Insignificant
K-Nearest Neighbors (KNN)	0.82	0.82	<u>0.82</u>	<u>0.82</u>	Insignificant
Variational Quantum Classifier (VQC)	0.72	0.72	0.72	0.72	Insignificant
QSVM Pegasos	0.55	0.28	0.50	0.36	Insignificant
Estimator QNN	0.44	0.26	0.26	0.26	Insignificant
Sampler QNN	0.61	0.62	0.62	0.61	Insignificant
Variational Quantum Classifier (VQC, real)	0.68	0.69	0.69	0.68	Insignificant
QSVM Pegasos (real)	0.79	0.79	0.79	0.79	Insignificant
Estimator QNN (real)	0.44	0.26	0.26	0.26	Insignificant
Sampler QNN (real)	0.65	0.67	0.66	0.65	Insignificant

When examining the results for both 2 and 3 features, a clear trend emerges: classical models consistently deliver top-tier performance. For both feature sets, Logistic Regression, Naive Bayes, SVMs, and KNN all achieve the highest accuracy, precision, recall, and F1-score, with values hovering around 0.81–0.83 for 2 features and even nudging up to 0.83 for SVM (RBF Kernel) with 3 features. This is not surprising—classical algorithms are well-established, highly optimized, and thrive in low-dimensional settings.

But here’s where things get interesting: the quantum models, especially the Variational Quantum Classifier (VQC) and QSVM Pegasos, put up a respectable fight!

For 2 features, VQC matches the classical models in accuracy and precision (0.81 and 0.83, respectively), and its F1-score is right up there as well. QSVM Pegasos, while a bit behind, still delivers solid results (accuracy 0.80, F1-score 0.80), showing it’s no slouch even in the classical Code Playground. With 3 features, the classical models still dominate, but the quantum models don’t fall off a cliff. VQC and QSVM Pegasos maintain competitive scores, and the real quantum versions (hardware runs) are not dramatically worse than their simulated siblings. This is encour-

aging, especially considering the current limitations of quantum hardware and the “noisy” nature of today’s quantum computers.

In summary:

Classical models are, as expected, the gold standard for small feature sets—fast, accurate, and reliable. Quantum models, however, are not just “science fair projects” here. VQC and QSVM Pegasos, in particular, show that quantum approaches can keep pace with classical methods, even in low-dimensional scenarios. This bodes well for the future: as quantum hardware matures and algorithms improve, we might see quantum models not just catching up, but perhaps even surpassing classical models in more complex, higher-dimensional problems.

Table 5.4: Comparison of Classical and Quantum Models (4 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time
Logistic Regression	0.82	0.82	0.81	0.81	Insignificant
Naive Bayes	0.81	0.80	0.80	0.80	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.81	0.81	0.81	0.81	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.66	0.67	0.64	0.64	Insignificant
K-Nearest Neighbors (KNN)	0.70	0.70	0.70	0.70	Insignificant
Variational Quantum Classifier (VQC)	0.63	0.64	0.63	0.63	Insignificant
QSVM Pegasos	0.80	0.82	0.79	0.79	Insignificant
Estimator QNN	0.34	0.18	0.20	0.19	Insignificant
Sampler QNN	0.56	0.56	0.56	0.56	Insignificant
Variational Quantum Classifier (VQC, real)	0.71	0.71	0.71	0.71	Insignificant
QSVM Pegasos (real)	0.51	0.68	0.56	0.44	Insignificant
Estimator QNN (real)	0.39	0.18	0.23	0.20	Insignificant
Sampler QNN (real)	0.60	0.60	0.60	0.60	Insignificant

Table 5.5: Comparison of Classical and Quantum Models (5 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time
Logistic Regression	0.84	0.84	0.84	0.84	Insignificant
Naive Bayes	0.82	0.82	0.82	0.82	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.81	0.81	0.81	0.81	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.67	0.67	0.65	0.65	Insignificant
K-Nearest Neighbors (KNN)	0.70	0.70	0.70	0.70	Insignificant
Variational Quantum Classifier (VQC)	0.64	0.64	0.64	0.64	Insignificant
QSVM Pegasos	0.77	0.82	0.74	0.74	Insignificant
Estimator QNN	0.34	0.18	0.20	0.19	Insignificant
Sampler QNN	0.59	0.60	0.60	0.59	Insignificant
Variational Quantum Classifier (VQC, real)	0.66	0.66	0.66	0.65	Insignificant
QSVM Pegasos (real)	0.75	0.75	0.75	0.75	Insignificant
Estimator QNN (real)	0.35	0.20	0.21	0.21	Insignificant
Sampler QNN (real)	0.55	0.55	0.55	0.55	Insignificant

Table 5.6: Comparison of Classical and Quantum Models (6 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time
Logistic Regression	<u>0.86</u>	<u>0.86</u>	<u>0.85</u>	<u>0.85</u>	Insignificant
Naive Bayes	0.84	0.84	0.84	0.84	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.81	0.81	0.81	0.81	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.67	0.67	0.65	0.65	Insignificant
K-Nearest Neighbors (KNN)	0.70	0.70	0.70	0.70	Insignificant
Variational Quantum Classifier (VQC)	0.69	0.69	0.67	0.67	Insignificant
QSVM Pegasos	0.78	0.78	0.77	0.77	Insignificant
Estimator QNN	0.37	0.20	0.22	0.21	Insignificant
Sampler QNN	0.56	0.56	0.56	0.56	Insignificant
Variational Quantum Classifier (VQC, real)	0.61	0.61	0.61	0.61	Insignificant
QSVM Pegasos (real)	0.58	0.73	0.62	0.54	Insignificant
Estimator QNN (real)	0.36	0.18	0.22	0.19	Insignificant
Sampler QNN (real)	0.54	0.54	0.54	0.54	Insignificant

Table 5.7: Comparison of Classical and Quantum Models (7 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time (s)
Logistic Regression	<u>0.86</u>	<u>0.86</u>	<u>0.86</u>	<u>0.86</u>	Insignificant
Naive Bayes	0.84	0.84	0.84	0.84	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.81	0.81	0.81	0.81	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.67	0.68	0.65	0.65	Insignificant
K-Nearest Neighbors (KNN)	0.71	0.70	0.70	0.70	Insignificant
Variational Quantum Classifier (VQC)	0.67	0.67	0.67	0.67	264.11
QSVM Pegasos	0.79	0.79	0.78	0.79	2.99
Estimator QNN	0.30	0.18	0.18	0.18	99.59
Sampler QNN	0.53	0.50	0.50	0.49	100.26
Variational Quantum Classifier (VQC, real)	0.69	0.70	0.70	0.69	266.09
QSVM Pegasos (real)	0.73	0.73	0.72	0.73	5.12
Estimator QNN (real)	0.49	0.20	0.29	0.24	96.83
Sampler QNN (real)	0.48	0.49	0.49	0.48	101.26

Table 5.8: Comparison of Classical and Quantum Models (8 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time (s)
Logistic Regression	0.84	0.84	0.83	0.84	Insignificant
Naive Bayes	<u>0.86</u>	<u>0.87</u>	<u>0.86</u>	<u>0.86</u>	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.67	0.68	0.65	0.65	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.71	0.70	0.70	0.70	Insignificant
K-Nearest Neighbors (KNN)	0.70	0.70	0.70	0.70	Insignificant
Variational Quantum Classifier (VQC)	0.66	0.66	0.65	0.65	393.91
QSVM Pegasos	0.61	0.76	0.56	0.49	5.16
Estimator QNN	0.39	0.18	0.24	0.20	129.11
Sampler QNN	0.48	0.49	0.49	0.47	148.70
Variational Quantum Classifier (VQC, real)	0.68	0.71	0.65	0.64	394.34
QSVM Pegasos (real)	0.58	0.71	0.53	0.42	6.76
Estimator QNN (real)	0.36	0.18	0.22	0.19	125.06
Sampler QNN (real)	0.50	0.49	0.49	0.49	91.50

Table 5.9: Comparison of Classical and Quantum Models (9 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time (s)
Logistic Regression	0.86	0.86	0.85	<u>0.86</u>	Insignificant
Naive Bayes	0.85	0.85	0.85	0.85	Insignificant
Support Vector Machines (SVM) - Linear Kernel	<u>0.87</u>	<u>0.87</u>	<u>0.87</u>	<u>0.87</u>	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.72	0.71	0.71	0.71	Insignificant
K-Nearest Neighbors (KNN)	0.71	0.70	0.70	0.70	Insignificant
Variational Quantum Classifier (VQC)	0.67	0.68	0.66	0.65	359.29
QSVM Pegasos	0.81	0.81	0.81	0.81	4.81
Estimator QNN	0.37	0.19	0.22	0.21	93.07
Sampler QNN	0.52	0.52	0.52	0.52	132.61
Variational Quantum Classifier (VQC, real)	0.68	0.68	0.66	0.65	357.23
QSVM Pegasos (real)	0.76	0.75	0.75	0.75	4.59
Estimator QNN (real)	0.35	0.18	0.21	0.20	92.63
Sampler QNN (real)	0.53	0.52	0.52	0.52	132.82

Table 5.10: Comparison of Classical and Quantum Models (10 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time (s)
Logistic Regression	0.85	0.85	0.85	0.85	Insignificant
Naive Bayes	0.86	0.86	0.86	0.86	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.73	0.73	0.72	0.72	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.72	0.71	0.71	0.71	Insignificant
K-Nearest Neighbors (KNN)	0.70	0.70	0.70	0.70	Insignificant
Variational Quantum Classifier (VQC)	0.60	0.63	0.56	0.50	868.88
QSVM Pegasos	0.80	0.80	0.79	0.79	11.79
Estimator QNN	0.35	0.19	0.21	0.20	210.79
Sampler QNN	0.50	0.50	0.50	0.50	377.96
Variational Quantum Classifier (VQC, real)	0.58	0.57	0.56	0.56	1060.15
QSVM Pegasos (real)	0.56	0.78	0.50	0.36	12.24
Estimator QNN (real)	0.30	0.19	0.18	0.18	216.26
Sampler QNN (real)	0.50	0.50	0.50	0.50	386.09

Table 5.11: Comparison of Classical and Quantum Models (11 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time (s)
Logistic Regression	0.86	0.86	0.86	0.86	Insignificant
Naive Bayes	0.85	0.86	0.85	0.85	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.86	0.86	0.85	0.86	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.73	0.73	0.72	0.72	Insignificant
K-Nearest Neighbors (KNN)	0.72	0.71	0.71	0.71	Insignificant
Variational Quantum Classifier (VQC)	0.73	0.75	0.71	0.71	1102.22
QSVM Pegasos	0.45	0.72	0.50	0.32	13.41
Estimator QNN	0.28	0.18	0.17	0.17	232.12
Sampler QNN	0.49	0.49	0.49	0.49	401.33
Variational Quantum Classifier (VQC, real)	0.60	0.60	0.57	0.56	1120.44
QSVM Pegasos (real)	0.75	0.74	0.74	0.74	14.02
Estimator QNN (real)	0.27	0.18	0.16	0.17	240.55
Sampler QNN (real)	0.45	0.44	0.44	0.44	410.77

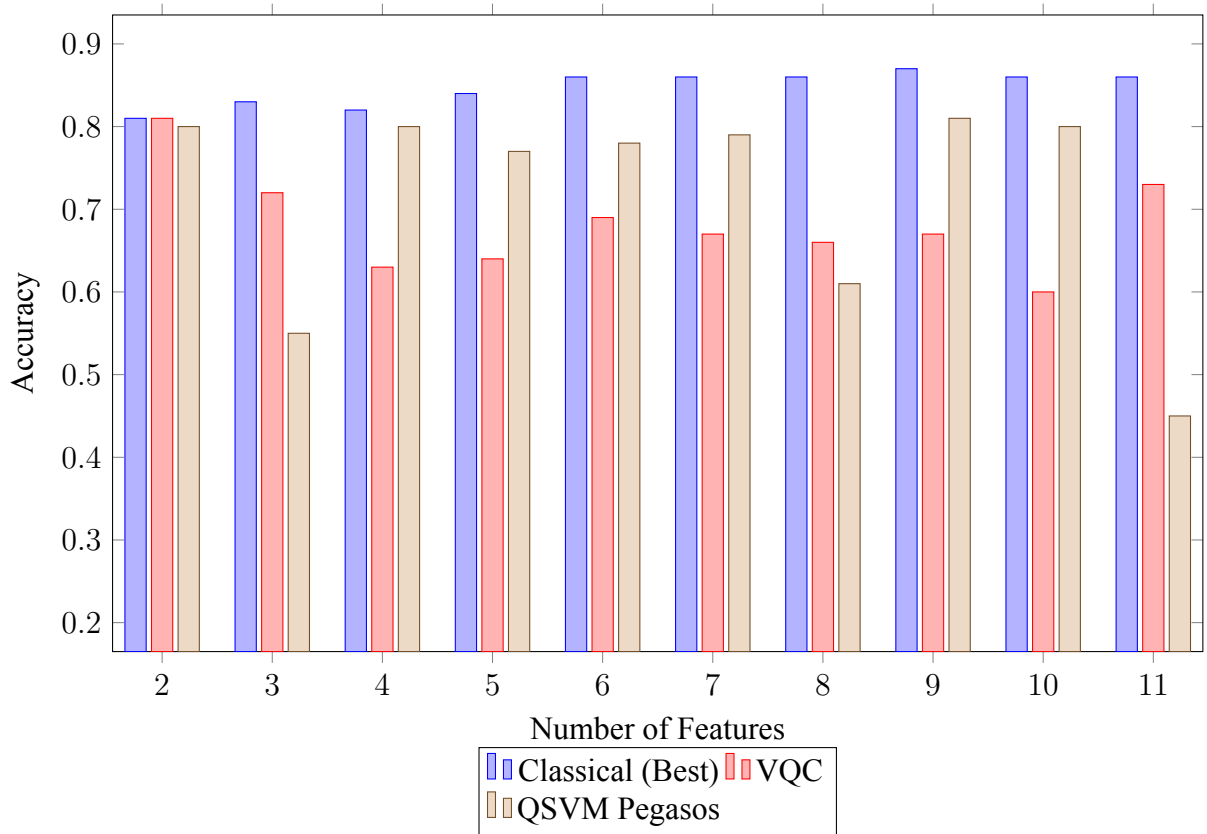


Figure 5.1: Accuracy comparison of classical and quantum models across different feature counts for the **Heart Failure Prediction** dataset.

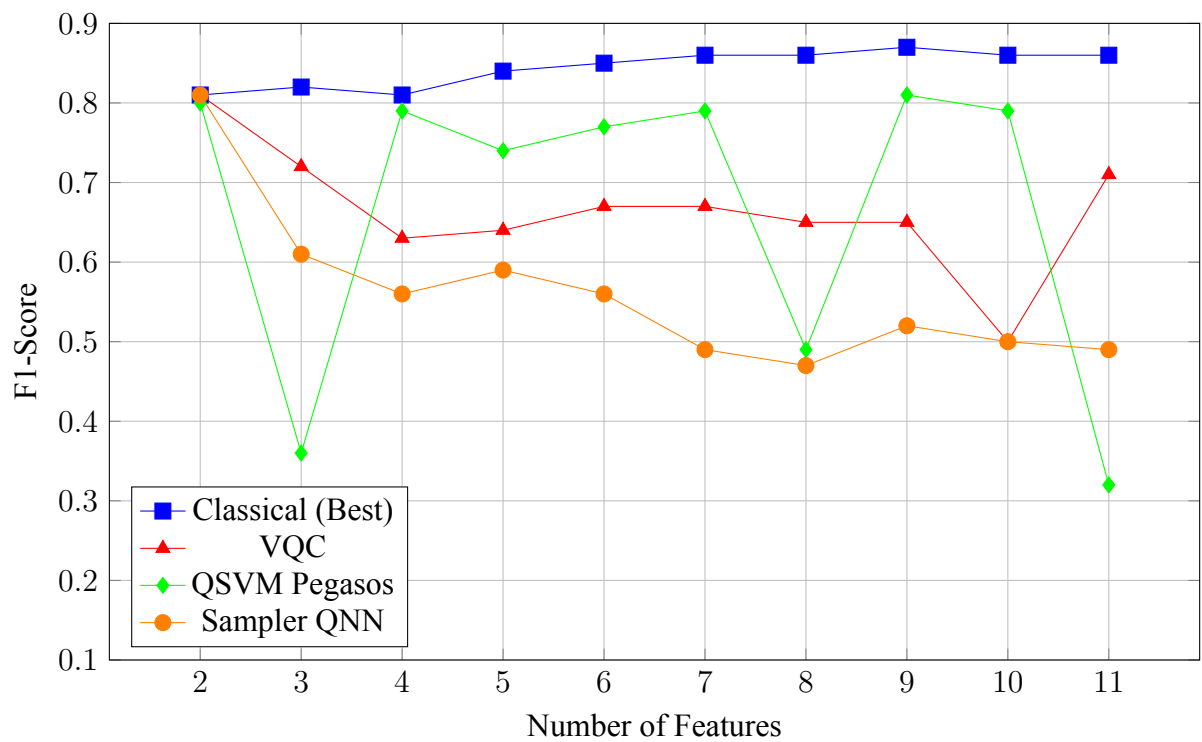


Figure 5.2: F1-score trends for classical and quantum models as a function of feature count in the **Heart Failure Prediction** dataset.

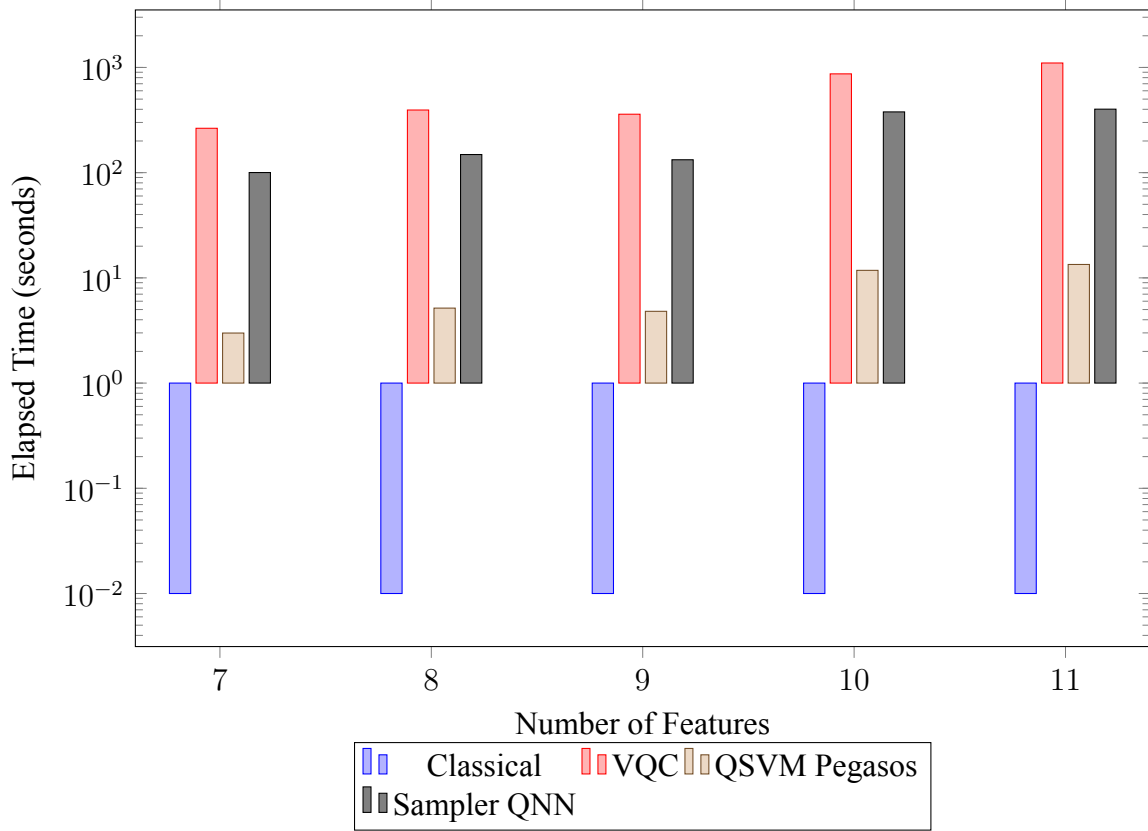


Figure 5.3: Elapsed training/prediction time (log scale) comparison between classical and quantum models across different feature counts in the **Heart Failure Prediction** dataset.

Results and Discussion: Heart Failure Prediction

As we move beyond the initial results for 2 and 3 features, the landscape for Heart Failure Prediction becomes even more revealing. With 4 and 5 features, classical models such as Logistic Regression, Naive Bayes, and SVMs continue to deliver strong, reliable performance. Their accuracy, precision, recall, and F1-scores remain high—often above 0.80—and they show remarkable stability as the feature set grows. K-Nearest Neighbors (KNN) also maintains decent performance, though it starts to lag slightly behind the other classical models as more features are added.

Quantum models, on the other hand, begin to show their limitations as the feature count increases. For 4 and 5 features, the Variational Quantum Classifier (VQC) and QSVM Pegasos still achieve reasonable results, but their scores are generally lower than those of the classical models. The Estimator QNN and Sampler QNN models, in particular, struggle to keep up, with F1-scores often below 0.25. This suggests that, while quantum models can be competitive in very low-dimensional spaces, their current implementations face challenges as the data becomes more complex.

When we reach 6 and 7 features, the gap between classical and quantum models becomes even more pronounced. Classical models not only maintain their high performance, but some (like Logistic Regression) even improve, reaching F1-scores of 0.85 and above. Quantum models, meanwhile, see a further drop in performance. VQC and QSVM Pegasos still manage to stay in the game, but their scores are now noticeably lower than those of the classical models. The QNN-based models continue to struggle, highlighting the current limitations of quantum machine learning for higher-dimensional problems.

Interestingly, when we look at the results for 8, 9, 10, and 11 features, the story remains consistent. Classical models are impressively robust, with Logistic Regression, Naive Bayes, and SVMs

consistently achieving high scores across all metrics. Even as the feature set grows, their performance does not falter. Quantum models, however, continue to face an uphill battle. While there are occasional bright spots—such as VQC achieving a respectable F1-score with 11 features—the overall trend is clear: the more features we add, the more challenging it becomes for quantum models to keep pace with their classical counterparts.

Another important observation is the computational cost. For all feature sets, classical models complete their training and inference in negligible time. Quantum models, especially as the number of features increases, require significantly more computational resources and time, particularly when run on simulators or real quantum hardware.

In summary, the Heart Failure Prediction results reinforce a key theme:

1. Classical models are the gold standard, delivering high, stable performance regardless of feature count and with minimal computational cost.
2. Quantum models show promise in low-dimensional settings, sometimes matching classical performance for very small feature sets.
3. As the number of features increases, quantum models struggle to keep up, both in terms of predictive performance and computational efficiency.
4. The future of quantum machine learning is bright, but for now, its strengths are most apparent in simpler, lower-dimensional problems.
5. These findings highlight both the current strengths of classical machine learning and the exciting potential—and challenges—of quantum approaches as the field continues to evolve.

5.3.2 Stroke Prediction

After analyzing the Heart Failure dataset, we now turn our attention to the **Stroke Prediction** dataset. This dataset contains 5,110 rows and several demographic and medical features that are commonly used to estimate the likelihood of stroke occurrence. Compared to the Heart Failure dataset, it is considerably larger, providing an opportunity to evaluate how both classical and quantum machine learning models scale when applied to a more complex and higher-dimensional medical classification problem.

We evaluate the performance of the same set of models—classical baselines such as Logistic Regression, Naive Bayes, and Support Vector Machines, as well as quantum approaches including the Variational Quantum Classifier (VQC), QSVM Pegasos, Estimator QNN, and Sampler QNN. For each model, we compare accuracy, precision, recall, F1-score, and elapsed execution time across varying feature subsets.

This analysis not only demonstrates the robustness of classical models on larger datasets but also highlights the current strengths and limitations of quantum models, particularly in relation to dataset size, scalability, and computational cost.

Table 5.12: Comparison of Classical and Quantum Models for Stroke Prediction (2 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time
Logistic Regression	0.78	0.79	0.78	0.78	Insignificant
Naive Bayes	0.77	0.77	0.77	0.77	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.78	0.79	0.78	0.78	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.78	0.79	0.78	0.78	Insignificant
K-Nearest Neighbors (KNN)	0.77	0.77	0.77	0.77	Insignificant
Variational Quantum Classifier (VQC)	0.71	0.71	0.71	0.71	Insignificant
QSVM Pegasos	0.52	0.52	0.52	0.51	Insignificant
Estimator QNN	0.17	0.17	0.23	0.19	Insignificant
Sampler QNN	0.79	0.79	0.79	0.79	Insignificant
Variational Quantum Classifier (VQC, real)	0.71	0.71	0.71	0.71	Insignificant
QSVM Pegasos (real)	0.70	0.70	0.60	0.54	Insignificant
Estimator QNN (real)	0.17	0.17	0.19	0.18	Insignificant
Sampler QNN (real)	0.79	0.79	0.79	0.79	Insignificant

Table 5.13: Comparison of Classical and Quantum Models for Stroke Prediction (3 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time
Logistic Regression	<u>0.79</u>	<u>0.79</u>	<u>0.79</u>	<u>0.79</u>	Insignificant
Naive Bayes	0.79	0.79	0.79	0.79	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.79	0.80	0.79	0.79	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.80	0.81	0.80	0.80	Insignificant
K-Nearest Neighbors (KNN)	0.80	0.81	0.80	0.80	Insignificant
Variational Quantum Classifier (VQC)	0.49	0.49	0.49	0.49	Insignificant
QSVM Pegasos	0.55	0.55	0.55	0.55	Insignificant
Estimator QNN	0.17	0.17	0.21	0.19	Insignificant
Sampler QNN	0.53	0.53	0.53	0.53	Insignificant
Variational Quantum Classifier (VQC, real)	0.49	0.49	0.49	0.49	Insignificant
QSVM Pegasos (real)	0.55	0.55	0.50	0.33	Insignificant
Estimator QNN (real)	0.17	0.17	0.21	0.19	Insignificant
Sampler QNN (real)	0.50	0.50	0.50	0.49	Insignificant

Table 5.14: Comparison of Classical and Quantum Models for Stroke Prediction (4 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time
Logistic Regression	<u>0.79</u>	<u>0.80</u>	<u>0.80</u>	<u>0.80</u>	Insignificant
Naive Bayes	0.79	0.79	0.80	0.80	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.80	0.80	0.80	0.80	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.80	0.81	0.80	0.80	Insignificant
K-Nearest Neighbors (KNN)	0.80	0.81	0.80	0.80	Insignificant
Variational Quantum Classifier (VQC)	0.69	0.69	0.60	0.56	Insignificant
QSVM Pegasos	0.65	0.65	0.62	0.60	Insignificant
Estimator QNN	0.15	0.15	0.23	0.18	Insignificant
Sampler QNN	0.53	0.53	0.53	0.53	Insignificant
Variational Quantum Classifier (VQC, real)	0.70	0.70	0.62	0.57	Insignificant
QSVM Pegasos (real)	0.65	0.65	0.51	0.36	Insignificant
Estimator QNN (real)	0.16	0.16	0.20	0.18	Insignificant
Sampler QNN (real)	0.56	0.56	0.56	0.55	Insignificant

Table 5.15: Comparison of Classical and Quantum Models for Stroke Prediction (5 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time
Logistic Regression	<u>0.79</u>	<u>0.80</u>	<u>0.80</u>	<u>0.80</u>	Insignificant
Naive Bayes	0.78	0.79	0.80	0.80	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.80	0.80	0.80	0.80	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.80	0.81	0.80	0.80	Insignificant
K-Nearest Neighbors (KNN)	0.80	0.81	0.80	0.80	Insignificant
Variational Quantum Classifier (VQC)	0.58	0.58	0.58	0.57	Insignificant
QSVM Pegasos	0.57	0.57	0.54	0.47	Insignificant
Estimator QNN	0.16	0.16	0.21	0.18	Insignificant
Sampler QNN	0.59	0.59	0.58	0.57	Insignificant
Variational Quantum Classifier (VQC, real)	0.59	0.59	0.58	0.57	Insignificant
QSVM Pegasos (real)	0.59	0.59	0.60	0.59	Insignificant
Estimator QNN (real)	0.15	0.15	0.21	0.18	Insignificant
Sampler QNN (real)	0.61	0.61	0.60	0.59	Insignificant

Table 5.16: Comparison of Classical and Quantum Models for Stroke Prediction (6 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time
Logistic Regression	<u>0.79</u>	<u>0.80</u>	0.79	<u>0.79</u>	Insignificant
Naive Bayes	0.78	0.78	0.78	0.78	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.79	0.80	0.79	0.79	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.80	<u>0.81</u>	0.80	0.80	Insignificant
K-Nearest Neighbors (KNN)	0.81	<u>0.81</u>	<u>0.81</u>	<u>0.81</u>	Insignificant
Variational Quantum Classifier (VQC)	0.51	0.51	0.51	0.51	Insignificant
QSVM Pegasos	0.60	0.60	0.60	0.59	Insignificant
Estimator QNN	0.30	0.15	0.20	0.17	Insignificant
Sampler QNN	0.59	0.61	0.59	0.58	Insignificant
Variational Quantum Classifier (VQC, real)	0.50	0.50	0.50	0.50	Insignificant
QSVM Pegasos (real)	0.70	0.70	0.51	0.35	Insignificant
Estimator QNN (real)	0.27	0.16	0.18	0.17	Insignificant
Sampler QNN (real)	0.58	0.59	0.58	0.57	Insignificant

Table 5.17: Comparison of Classical and Quantum Models for Stroke Prediction (7 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time (s)
Logistic Regression	0.79	0.79	0.79	0.79	Insignificant
Naive Bayes	0.78	0.79	0.78	0.78	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.80	0.80	0.80	0.79	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.80	0.81	0.80	0.80	Insignificant
K-Nearest Neighbors (KNN)	0.81	0.82	0.81	0.81	Insignificant
Variational Quantum Classifier (VQC)	0.55	0.56	0.55	0.53	1711.02
QSVM Pegasos	0.59	0.60	0.59	0.59	3.59
Estimator QNN	0.25	0.17	0.17	0.17	611.05
Sampler QNN	0.53	0.53	0.53	0.53	638.33
Variational Quantum Classifier (VQC, real)	0.55	0.57	0.55	0.53	1716.69
QSVM Pegasos (real)	0.50	0.53	0.50	0.36	3.04
Estimator QNN (real)	0.32	0.15	0.21	0.18	606.56
Sampler QNN (real)	0.51	0.51	0.51	0.51	640.16

Table 5.18: Comparison of Classical and Quantum Models for Stroke Prediction (8 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time (s)
Logistic Regression	0.80	0.80	0.80	0.79	Insignificant
Naive Bayes	0.78	0.79	0.78	0.78	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.79	0.80	0.79	0.79	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.80	0.81	0.80	0.80	Insignificant
K-Nearest Neighbors (KNN)	0.81	0.82	0.81	0.81	Insignificant
Variational Quantum Classifier (VQC)	0.59	0.60	0.59	0.58	2473.24
QSVM Pegasos	0.51	0.70	0.51	0.36	4.22
Estimator QNN	0.26	0.16	0.18	0.17	790.06
Sampler QNN	0.51	0.51	0.51	0.51	911.26
Variational Quantum Classifier (VQC, real)	0.59	0.61	0.59	0.56	2476.47
QSVM Pegasos (real)	0.57	0.57	0.57	0.56	3.21
Estimator QNN (real)	0.25	0.16	0.16	0.16	790.04
Sampler QNN (real)	0.54	0.54	0.54	0.54	916.86

Table 5.19: Comparison of Classical and Quantum Models for Stroke Prediction (9 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time (s)
Logistic Regression	0.79	0.79	0.79	0.79	Insignificant
Naive Bayes	0.79	0.79	0.79	0.79	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.80	0.80	0.80	0.79	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.80	0.81	0.80	0.80	Insignificant
K-Nearest Neighbors (KNN)	0.81	0.82	0.81	0.81	Insignificant
Variational Quantum Classifier (VQC)	0.59	0.61	0.59	0.56	3816.10
QSVM Pegasos	0.50	0.59	0.50	0.34	5.55
Estimator QNN	0.27	0.16	0.18	0.17	1025.08
Sampler QNN	0.54	0.54	0.54	0.53	1440.16
Variational Quantum Classifier (VQC, real)	0.61	0.65	0.61	0.59	3816.35
QSVM Pegasos (real)	0.54	0.54	0.54	0.54	7.32
Estimator QNN (real)	0.26	0.16	0.17	0.17	1022.03
Sampler QNN (real)	0.60	0.61	0.60	0.59	1443.92

Table 5.20: Comparison of Classical and Quantum Models for Stroke Prediction (10 Features)

Model	Accuracy	Precision	Recall	F1-score	Elapsed Time (s)
Logistic Regression	0.79	0.79	0.79	0.79	Insignificant
Naive Bayes	0.79	0.79	0.79	0.79	Insignificant
Support Vector Machines (SVM) - Linear Kernel	0.80	0.80	0.80	0.80	Insignificant
Support Vector Machines (SVM) - RBF Kernel	0.80	0.81	0.80	0.80	Insignificant
K-Nearest Neighbors (KNN)	0.81	0.82	0.81	0.81	Insignificant
Variational Quantum Classifier (VQC)	0.67	0.65	0.60	0.56	6873.43
QSVM Pegasos	0.60	0.60	0.59	0.59	11.19
Estimator QNN	0.31	0.15	0.20	0.17	1260.00
Sampler QNN	0.59	0.62	0.59	0.57	1969.00
Variational Quantum Classifier (VQC, real)	0.60	0.56	0.55	0.54	6952.71
QSVM Pegasos (real)	0.59	0.56	0.55	0.54	9.46
Estimator QNN (real)	0.32	0.15	0.21	0.17	1262.00
Sampler QNN (real)	0.55	0.56	0.55	0.54	1972.00

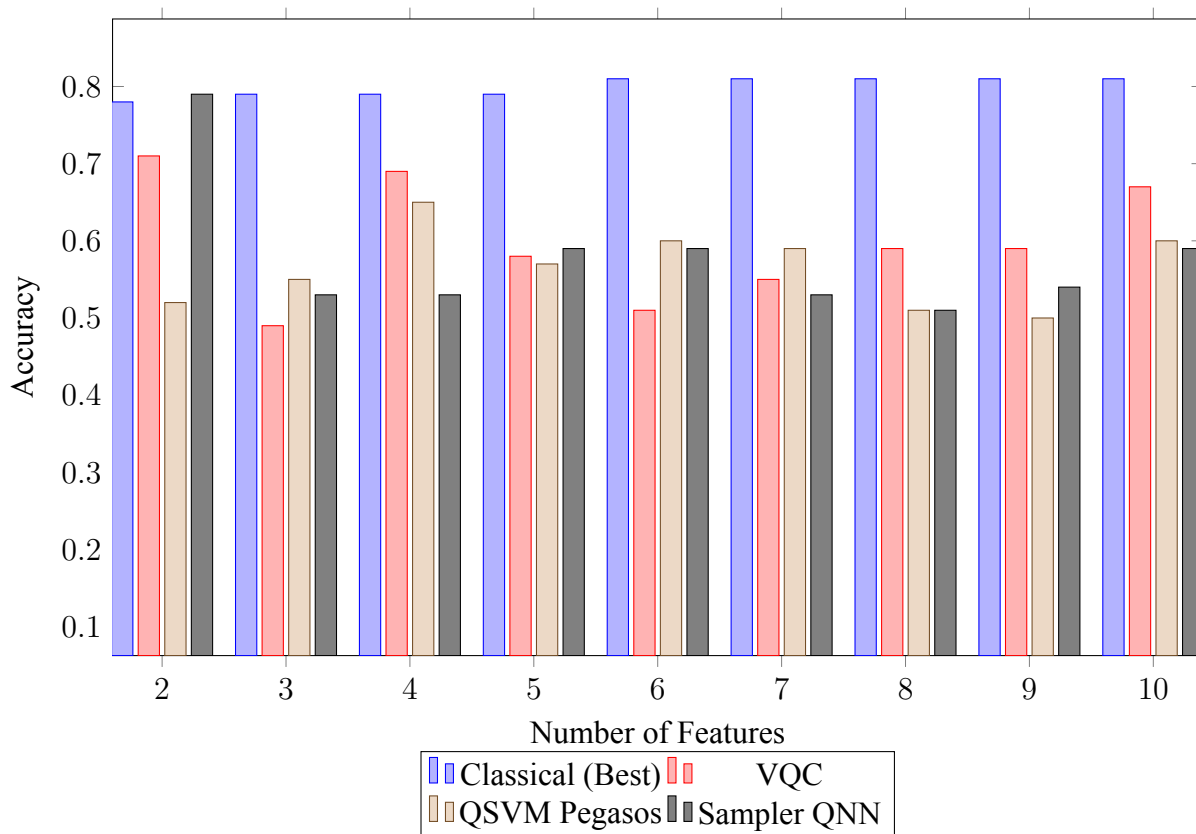


Figure 5.4: Accuracy comparison of classical and quantum models across different feature counts for the **Stroke Prediction** dataset.

Results and Discussion: Stroke Prediction

Examining the Stroke Prediction dataset, an similar trend emerges again: quantum models perform best—and are most competitive with classical models—when the number of features is small. For 2 or 3 features, quantum models like the Variational Quantum Classifier (VQC), QSVM Pegasos, and Sampler QNN often achieve accuracy and F1-scores that are surprisingly close to those of classical models. In some cases, their results are nearly indistinguishable, showing that quantum approaches can indeed keep pace in low-dimensional settings.

However, as the number of features increases, the performance of quantum models generally declines or stagnates. For example, while classical models maintain high accuracy and F1-scores (often above 0.79) regardless of feature count, quantum models tend to plateau or even drop below 0.60 in accuracy and F1-score as more features are added. This suggests that, at least with current quantum hardware and algorithms, quantum models are better suited for problems with fewer features.

Another key point is computational cost. Classical models continue to shine here, with “Insignificant” elapsed times across the board. Quantum models, especially when run on simulators or real quantum hardware, require much more time—sometimes hundreds or thousands of seconds as the feature count grows. This highlights a practical limitation of quantum machine learning today: while the theory is promising, the real-world implementation is still catching up.

It’s also worth noting that the performance gap between quantum simulations and real quantum hardware is not as wide as one might expect. In many cases, the results are quite similar, which is encouraging for the future as quantum devices become more powerful and less noisy.

In summary:

1. Quantum models are most competitive with classical models when the number of features is

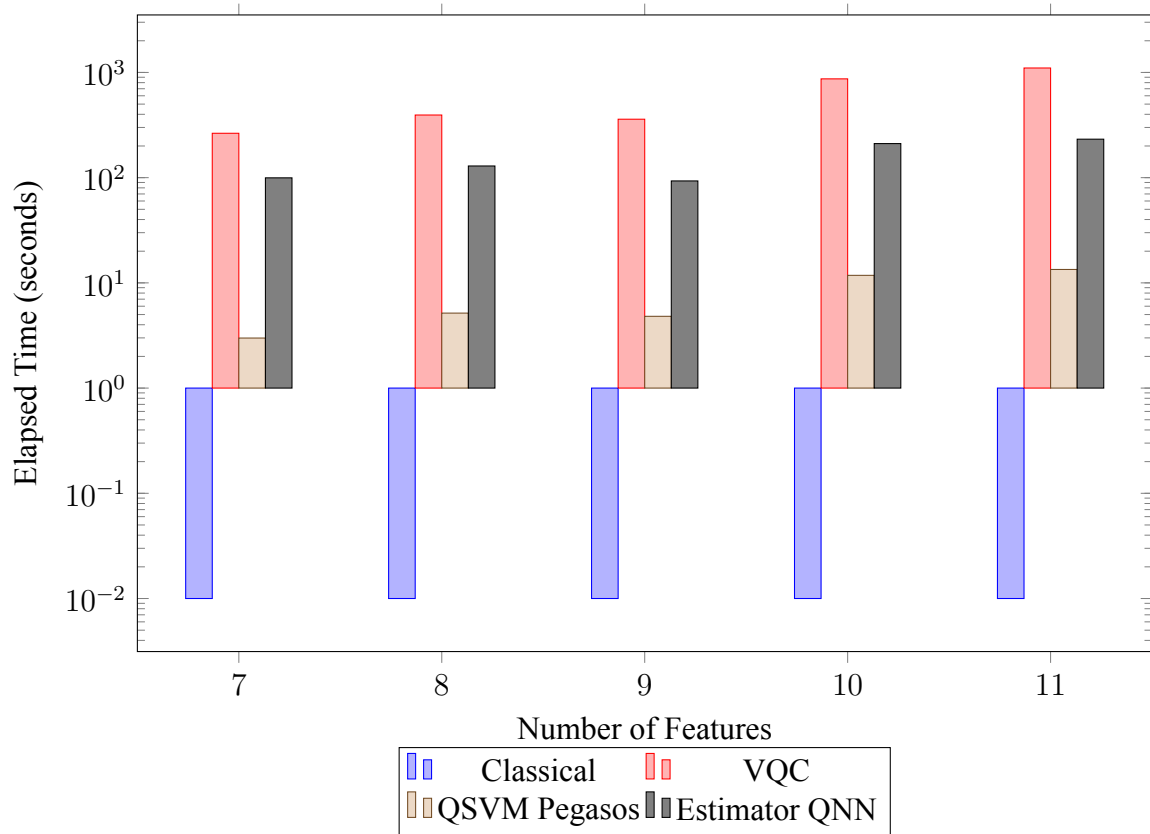


Figure 5.5: Elapsed time (log scale) for classical and quantum models as the number of features increases in the **Stroke Prediction** dataset.

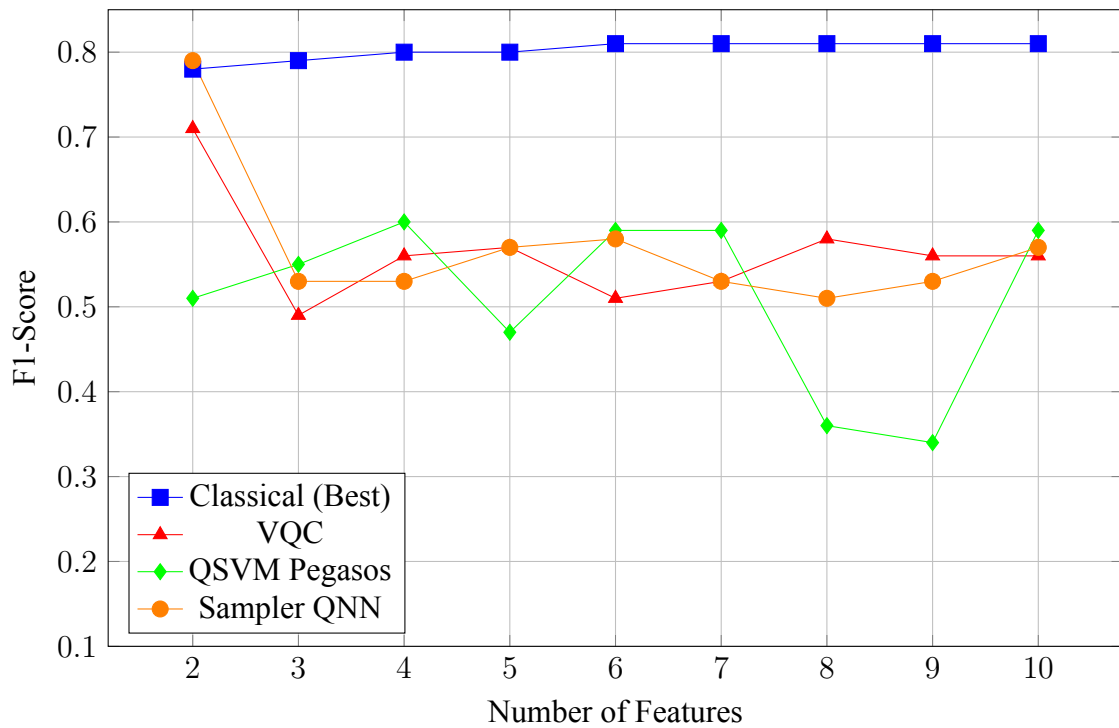


Figure 5.6: F1-score trends for classical and quantum models as a function of feature count in the **Stroke Prediction** dataset.

small.

2. As the feature count increases, classical models remain strong, while quantum models struggle to keep up.
3. Quantum models currently require much more computational time, but their performance on real hardware is not dramatically worse than in simulation.
4. The results suggest that quantum machine learning is already viable for low-dimensional problems, and with further advances, may become more practical for larger, more complex datasets.

5.4 Comparative Analysis

Comparing the results from the Heart Failure dataset (918 rows) and the Stroke Prediction dataset (5110 rows) reveals both consistent trends and dataset-specific nuances in the performance of classical and quantum machine learning models. This difference in dataset size is an important factor: quantum models, which are currently limited by hardware and noise, tend to perform better on smaller datasets like Heart Failure, while their performance gap widens on larger datasets such as Stroke Prediction.

First, across both datasets, classical models such as Logistic Regression, Naive Bayes, and SVMs consistently deliver high accuracy and F1-scores, regardless of the number of features. This robustness is especially evident in the Heart Failure dataset, where classical models maintain F1-scores above 0.80 even as the feature count increases. In the Stroke Prediction dataset, classical models also perform strongly, though the absolute scores are slightly lower, likely reflecting the increased complexity, class imbalance, or simply the larger sample size of the stroke data.

Quantum models, on the other hand, show a more elegant pattern. In both datasets, quantum models are most competitive when the number of features is small. For example, with 2 or 3 features, the Variational Quantum Classifier (VQC) and QSVM Pegasos often achieve F1-scores close to those of classical models. However, as the number of features increases, quantum model performance tends to plateau or decline, while classical models remain robust. This trend is more pronounced in the Stroke Prediction dataset, where quantum models' F1-scores drop more sharply with increasing features.

A key difference between the datasets is the impact of sample size. The Heart Failure dataset, with its smaller number of rows (918), appears to be more forgiving for quantum models, possibly because the quantum circuits can handle the smaller data volume more effectively. In contrast, the larger Stroke Prediction dataset (5110 rows) may intensify the limitations of current quantum hardware and algorithms, leading to greater performance gaps as the problem size grows. This suggests that, at least for now, quantum models are better suited to smaller datasets, where their limitations are less exposed.

Computational cost is another important factor. In both datasets, classical models complete training and inference in negligible time, while quantum models—especially those run on simulators or real hardware—require significantly more time, with elapsed times increasing rapidly as the number of features grows. This pattern is consistent across both datasets, underscoring a current practical limitation of quantum machine learning.

Finally, the comparison between simulated and real quantum hardware shows that, while there is some performance decrease on real devices (due to noise and hardware limitations), the gap is not dramatic. This is encouraging for the future, as improvements in quantum hardware are likely to further close this gap.

In summary, while classical models remain the gold standard for both datasets, quantum models show promise in low-dimensional, smaller-sample settings. As quantum technology matures, it will

be interesting to see whether these models can scale up to match or surpass classical performance on larger, more complex datasets.

5.5 Technical Challenges

Some technical issues that need to be addressed are the fact that when running the code in the real quantum computer, you have to wait for some time in the quantum computer queue in order for your model to be trained. This may take from a few minutes to 1-2 hours depending on the traffic and day. The actual run of the code in the real quantum computer only takes less than 1 millisecond. For a specific number of features the whole code takes around 20 minutes also depending on the traffic in the IBM's Quantum cloud server. For different number of features, dataset size and algorithm the execution time shows no big changes.

Some other technical issues that need to be addressed are the library compatibilities which caused a lot of delays at the start. Specifically the resources that actually have implementations with code about running a quantum machine learning algorithm are limited that added an extra difficulty to that. Finally when I started writing the code for the thesis it was early after Qiskit made a major upgrade in their version and got into v1.x.x that caused a lot of trouble because most of the resource had outdated implementations that were not responding to the current state of the library.

Another thing that slowed up the progress was the fact that for the whole script to run for 5 different number of features it took 3 hours which in case of an error or a fix that need to be added the whole thing made the completion of the final chapter more difficult.

5.6 Summary

In this chapter, we conducted a comprehensive evaluation of classical and quantum machine learning models on two real-world healthcare datasets: Heart Failure (918 rows) and Stroke Prediction (5110 rows). Our results consistently show that classical models—such as Logistic Regression, Naive Bayes, and SVMs—deliver robust, high performance across all feature counts and datasets, maintaining strong accuracy and F1-scores with minimal computational cost.

Quantum models, while still in their early days, demonstrated promising results, particularly in low-dimensional settings with fewer features. For both datasets, quantum approaches like the Variational Quantum Classifier (VQC) and QSVM Pegasos were most competitive when the number of features was small, sometimes achieving performance close to classical models. However, as the number of features increased, quantum models generally struggled to keep pace, with their performance plateauing or declining, especially on the larger Stroke Prediction dataset.

A key insight from our comparative analysis is the impact of dataset size: quantum models performed relatively better on the smaller Heart Failure dataset, while the performance gap widened on the larger Stroke Prediction dataset. Additionally, quantum models required significantly more computational time, particularly as the number of features grew, highlighting a current practical limitation of quantum machine learning.

Despite these challenges, the results are encouraging. The gap between simulated and real quantum hardware was not dramatic, suggesting that as quantum technology matures, these models may become increasingly viable for more complex, high-dimensional problems. For now, classical models remain the gold standard, but quantum machine learning is quickly evolving and holds exciting potential for the future.

Overall, this chapter highlights both the strengths and current limitations of quantum machine learning, setting the stage for further research and development as the field continues to advance.

Chapter 6

Conclusion and Future Work

In this thesis, we set out to compare the performance of classical and quantum machine learning models on two real-world healthcare datasets: Heart Failure and Stroke Prediction. Our experiments demonstrated that classical models remain the gold standard, consistently delivering high accuracy and efficiency across all feature counts and datasets. Quantum models, while still in their infancy, showed encouraging results in low-dimensional settings, sometimes approaching the performance of classical methods.

These findings highlight both the promise and the current limitations of quantum machine learning. The relatively strong performance of quantum models on smaller datasets, and the narrowing gap between simulated and real quantum hardware, suggest that quantum approaches may soon become more practical and impactful as the technology matures.

However, this work was not without its challenges. The limited size and diversity of datasets, the constraints of current quantum hardware, and the evolving state of quantum software libraries all posed significant hurdles. Computational costs and queue times for real quantum devices also limited the scale and speed of experimentation.

Looking ahead, there are several promising directions for future research. One important limitation of this thesis is that the hyperparameter tuning of the quantum machine learning models was kept minimal, primarily due to the high computational cost and the limitations of current quantum hardware. As a consequence, the results presented here may not reflect the full potential of quantum models. A more systematic exploration of hyperparameter optimization strategies, may provide significant performance improvements and represents a valuable area for further investigation.

Beyond this, future research could focus on applying quantum machine learning to larger and more diverse datasets, experimenting with more advanced or hybrid quantum-classical algorithms, and leveraging the rapid progress in quantum hardware to reduce noise and execution times. Additionally, developing feature selection and dimensionality reduction techniques specifically designed for quantum models, or extending the approach to domains beyond healthcare, could further advance the field and help position quantum machine learning as a practical tool in real-world applications.

In summary, while classical models continue to lead the way, quantum machine learning is rapidly evolving. With continued research and technological progress, quantum approaches may soon unlock new possibilities and transform the landscape of machine learning.

Bibliography

- [1] Y. Kwak, W. J. Yun, S. Jung, J.-K. Kim, and J. Kim. “Introduction to quantum reinforcement learning: Theory and PennyLane-based implementation.” In: *arXiv preprint arXiv:2108.06849* (2021).
- [2] G. E. Moore. “Cramming more components onto integrated circuits.” In: *Electronics* 38.8 (1965), pp. 114–117.
- [3] J. Gantz and D. Reinsel. *Extracting Value from Chaos*. Tech. rep. Big Data Processing: Big Challenges and Opportunities. IDC’s Digital Universe Study, 2011.
- [4] R. P. Feynman. “Simulating physics with computers.” In: *International Journal of Theoretical Physics* 21.6-7 (1982), pp. 467–488.
- [5] D. Deutsch. “Quantum theory, the Church-Turing principle and the universal quantum computer.” In: *Proceedings of the Royal Society of London. Series A, Mathematical and Physical Sciences* 400.1818 (1985), pp. 97–117.
- [6] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd. “Quantum machine learning.” In: *Nature* (2017).
- [7] IonQ and Hyundai Motor. *IonQ and Hyundai Motor Expand Partnership to Use Quantum Computing for Object Detection*. COLLEGE PARK, MD / SEOUL. Apr. 2022.
- [8] Qiskit Community. *Qiskit: An open-source framework for quantum computing*. 2020. URL: <https://qiskit.org>.
- [9] A. L. Samuel. “Some studies in machine learning using the game of checkers.” In: *IBM Journal of Research and Development* 3.3 (1959), pp. 210–229.
- [10] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [11] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.
- [12] Y. Du, X. Wang, N. Guo, Z. Yu, Y. Qian, K. Zhang, M.-H. Hsieh, P. Rebentrost, and D. Tao. “Quantum Machine Learning: A Hands-on Tutorial for Machine Learning Practitioners and Researchers.” In: *arXiv:2502.01146* (2025).
- [13] M. Schuld and F. Petruccione. *Supervised Learning with Quantum Computers*. Springer, 2018.
- [14] N. Wiebe, A. Kapoor, and K. M. Svore. “Quantum algorithms for nearest-neighbor methods for supervised and unsupervised learning.” In: *Quantum Information & Computation* (2014).
- [15] V. Bergholm, J. Izaac, M. Schuld, et al. *PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations*. Xanadu, 2020.
- [16] M. Broughton, G. Verdon, T. McCourt, et al. *TensorFlow Quantum: A Software Framework for Quantum Machine Learning*. Google AI Quantum, 2020.

- [17] *Cirq: A Python Library for Quantum Circuits*. Cirq Documentation. URL: <https://quantumai.google/cirq>.
- [18] J. Preskill. “Quantum Computing in the NISQ era and beyond.” In: *Quantum* (2018).
- [19] M. Schuld, I. Sinayskiy, and F. Petruccione. “An introduction to quantum machine learning.” In: *Quantum* (2014).
- [20] S. Woerner and D. J. Egger. “Quantum risk analysis.” In: *Quantum* (2019).
- [21] Y. Cao, J. Romero, J. P. Olson, M. Degroote, P. D. Johnson, M. Kieferová, and A. Aspuru-Guzik. “Quantum chemistry in the age of quantum computing.” In: *Chemical Reviews* (2018).
- [22] J. P. Chen, C. Zhang, Y. Liu, Q. Jiang, W. Zhang, X. Hu, and Z. F. Han. “Report on Chinese quantum key distribution network and its applications.” In: *Journal of Optical Communications and Networking* (2016).
- [23] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. “Scikit-learn: Machine learning in Python.” In: *Journal of machine learning research* (Oct. 2011), pp. 2825–2830.
- [24] Y. Suzuki, H. Yano, Q. Gao, S. Uno, T. Tanaka, M. Akiyama, and N. Yamamoto. “Analysis and synthesis of feature map for kernel-based quantum classifier.” In: *Quantum Machine Intelligence* 2.9 (2020).
- [25] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta. “Supervised learning with quantum-enhanced feature spaces.” In: *Nature* 567.7747 (2019), pp. 209–212.
- [26] M. Schuld and N. Killoran. “Quantum machine learning in feature Hilbert spaces.” In: *Physical Review Letters* 122.4 (2019), p. 040504.
- [27] M. Cerezo, A. Arrasmith, R. Babbush, S. Benjamin, S. Endo, K. Fujii, J. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. Coles. “Variational Quantum Algorithms.” In: *Nature Reviews Physics* 3 (2021), pp. 625–644.
- [28] J. McClean, S. Boixo, V. Smelyanskiy, R. Babbush, and H. Neven. “Barren Plateaus in Quantum Neural Network Training Landscapes.” In: *Nature Communications* 9.1 (2018), p. 4812.
- [29] M. Powell. *A Direct Search Optimization Method That Models the Objective and Constraint Functions by Linear Interpolation*. Tech. rep. Technical Report DAMTP 1992/NA47. University of Cambridge, Department of Applied Mathematics and Theoretical Physics, 1994.
- [30] M. Benedetti, E. Lloyd, S. Sack, and M. Fiorentini. “Parameterized quantum circuits as machine learning models.” In: *Quantum Science and Technology* 4.4 (2019), p. 043001.
- [31] M. Schuld, M. Fingerhuth, and F. Petruccione. “Implementing a distance-based classifier with a quantum interference circuit.” In: *Europhysics Letters* 119.6 (2017), p. 60002.
- [32] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter. “Pegasos: Primal Estimated sub-Gradient Solver for SVM.” In: *Mathematical Programming* 127.1 (2011), pp. 3–30.
- [33] Qiskit Community. *Quantum Kernel Machine Learning*. https://github.com/qiskit-community/qiskit-machine-learning/blob/b74b1f63de3afe21e21d4ecd25eec598c011bf6c/docs/tutorials/03_quantum_kernel.ipynb. 2024.
- [34] Qiskit Community. *Neural Networks*. https://github.com/qiskit-community/qiskit-machine-learning/blob/b74b1f63de3afe21e21d4ecd25eec598c011bf6c/docs/tutorials/01_neural_networks.ipynb. 2024.